Dossier Projet - RNCP 5

Titre professionnel : Développeur Web et Web Mobile (DWWM)

Page de garde

- **Titre du projet** : FloraShop — Pivoine & Lilas, Application e-commerce pour artisan fleuriste
- **Nom et prénom du candidat** : Jaille Dimitri
- **Date** : Juin 2025
- **Titre professionnel visé** : Développeur Web et Web Mobile (DWWM) - RNCP Niveau 5
- **Organisme de formation** : Holberton School
- **Session** : 2025-2026

Fig. 1 : Logo Holberton School

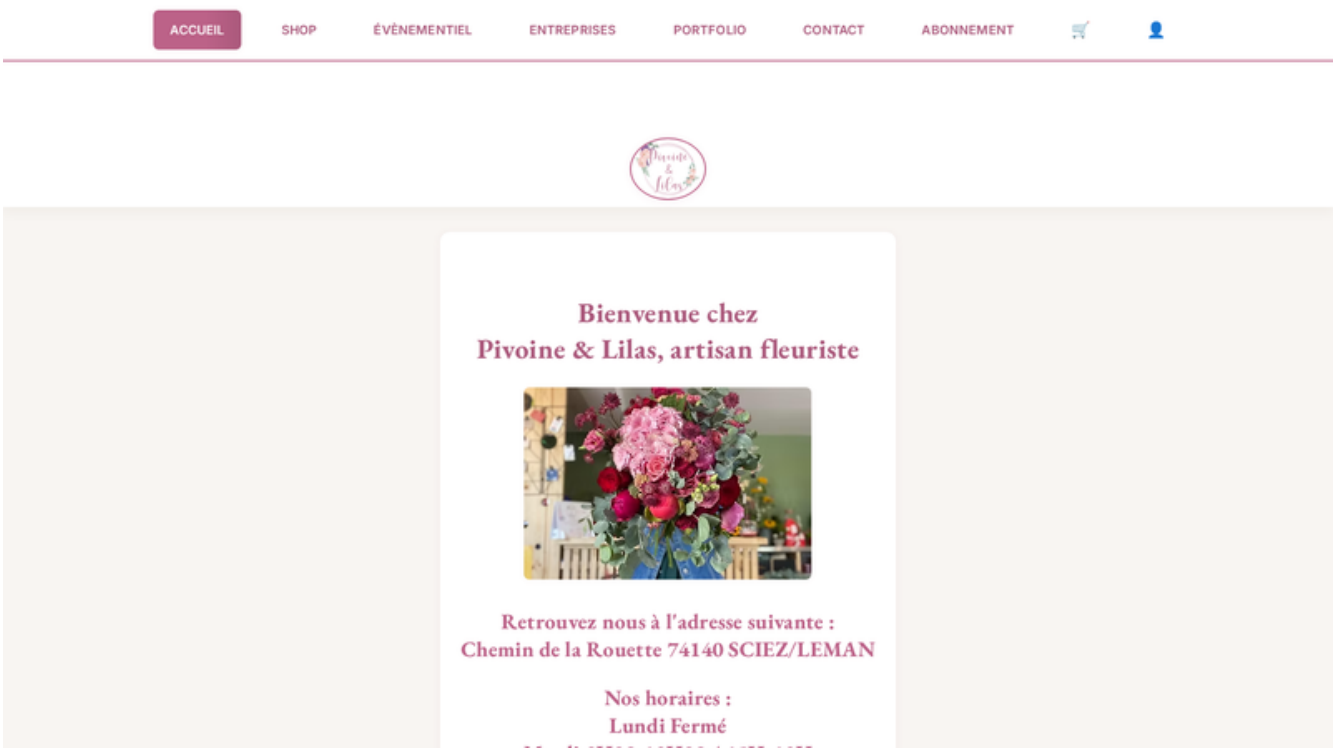


Fig. 2 : Page d'accueil FloraShop - Pivoine & Lilas

Remerciements

Je tiens à remercier l'ensemble de l'équipe pédagogique de Holberton School pour leur accompagnement tout au long de cette formation. Leurs conseils techniques et méthodologiques m'ont permis de progresser et de mener ce projet à bien.

Je remercie tout particulièrement la mère de Mattieu Mourroux, gérante de Pivoine & Lilas, pour sa confiance dans la réalisation de cette application e-commerce. Ce projet est né de la volonté d'aider son entreprise à développer son activité d'artisan fleuriste en lui offrant une présence en ligne professionnelle. Sa disponibilité lors de la définition du cahier des charges et ses retours sur les besoins fonctionnels ont été essentiels à la réussite du projet.

Merci à Mattieu Mourroux, ami et co-développeur sur ce projet, pour sa collaboration précieuse tout au long du développement.

Merci à mes camarades de promotion pour les échanges constructifs, le partage de connaissances et l'entraide qui ont contribué à enrichir mon apprentissage.

Enfin, je remercie mes proches pour leur soutien et leurs encouragements constants tout au long de cette formation.

Introduction et Contexte

Le projet FloraShop — Pivoine & Lilas s'inscrit dans une démarche de digitalisation d'une entreprise artisanale. L'objectif principal était de concevoir une solution e-commerce adaptée aux besoins spécifiques d'un fleuriste indépendant, en tenant compte de ses contraintes métier et de ses attentes en matière d'expérience utilisateur.

Analyse du besoin

L'entreprise Pivoine & Lilas, gérée par une artisane passionnée, souhaitait élargir sa clientèle et faciliter la gestion de ses ventes grâce à une boutique en ligne. Après plusieurs entretiens, il est apparu que la solution devait : - Permettre la mise en avant de créations florales uniques, avec des visuels attractifs - Offrir un parcours d'achat simple, rapide et sécurisé - Intégrer un système de gestion des stocks et des commandes adapté à une petite structure - Être administrable sans compétences techniques avancées

Choix techniques et argumentation

Le choix de technologies s'est porté sur : - **Front-end** : HTML5, CSS3, JavaScript vanilla pour garantir la légèreté, la rapidité et la personnalisation du design, tout en assurant la compatibilité multi-supports. - **Back-end** : Python Flask, pour sa simplicité, sa robustesse et la facilité d'intégration avec des outils comme Stripe et PostgreSQL. - **Base de données** : PostgreSQL, pour sa fiabilité et sa gestion avancée des transactions. - **Sécurité** : Authentification JWT, hashage des mots de passe, validation des entrées, afin de protéger les données sensibles des utilisateurs.

Chaque choix a été motivé par la volonté de répondre précisément aux besoins du client, tout en respectant les bonnes pratiques du développement web moderne.

Démarche de développement

Le projet a été mené en suivant une méthodologie agile : itérations courtes, retours réguliers du client, ajustements progressifs. Cette approche a permis d'identifier rapidement les points d'amélioration et d'adapter la solution en fonction des retours d'utilisation.

Table des matières

1. Liste des compétences du référentiel couvertes par le projet
2. Contexte du projet
   - 2.1 Type de cahier des charges
   - 2.2 Présentation de l'entreprise et du service
   - 2.3 Cahier des charges / Expression des besoins
   - 2.4 Contraintes du projet et livrables attendus
   - 2.5 Environnement humain, technique et objectifs de qualité

3. Éléments significatifs côté front-end
4. Éléments significatifs côté back-end
5. Conclusion
6. Bibliographie
7. Annexes
   - 7.1 Swagger UI (documentation API)
   - 7.2 Données initiales
   - 7.3 Historique des migrations
   - 7.4 Glossaire technique
   - 7.5 Structure du projet

Liste des compétences du référentiel couvertes par le projet

AT1 - Développer la partie front-end d'une application web ou web mobile sécurisée

☒ CP1 - Maquetter des interfaces utilisateur web ou web mobile
- Maquettes des pages : accueil, boutique, panier, checkout, administration, compte, inscription, contact, portfolio, événementiel, entreprises, abonnement
- Design responsive (desktop / tablette / mobile)

☒ CP2 - Réaliser des interfaces utilisateur statiques web ou web mobile
- 16 templates HTML5/CSS3 (Jinja2), 4 fichiers CSS (932 lignes), charte graphique vintage cohérente
- Navigation responsive avec barre fixe, logo, menu, icones panier et profil

☒ CP3 - Développer la partie dynamique des interfaces utilisateur web ou web mobile
- JavaScript vanilla (api.js : 261 lignes, auth.js : 59 lignes)
- Gestion dynamique du panier (ajout, suppression, calcul total), filtres produits, recherche, authentification côté client, intégration Stripe.js

AT2 - Développer la partie back-end d'une application web ou web mobile sécurisée

☒ CP4 - Mettre en place une base de données relationnelle
- PostgreSQL avec 7 tables : users, catégories, products, orders, order_items, reviews, prices
- Relations : Category→Products (1-N), Order→OrderItems (1-N), User→Orders (1-N), User→Reviews (1-N)
- Gestion des migrations avec Flask-Migrate (Alembic) : 6 versions de migration

☒ CP5 - Développer des composants d'accès aux données SQL et NoSQL
- Pattern Repository : CategoryRepository, ProductRepository, PriceRepository, UserRepository, ReviewRepository
- ORM SQLAlchemy avec opérations CRUD complètes, validation d'ID, vérification d'unicité
- Facade de services agrégeant tous les repositories

☒ CP6 - Développer des composants métier côté serveur
- REST API documentée via Flask-RESTX (Swagger UI auto-générée)
- StripeService : création de PaymentIntent, confirmation de paiement, gestion webhooks
- Authentification JWT (PyJWT), gestion des roles (admin/utilisateur)
- Règles métier : calcul de totaux, gestion des statuts de commande (pending/paid/failed/cancel

---

Contexte du projet

Type de cahier des charges

☒ Projet en entreprise
☐ Projet en formation

Présentation de l'entreprise et du service

Le projet FloraShop a été réalisé dans le cadre de la formation Développeur Web et Web Mobile (DWWM) à Holberton School pour le compte de l'entreprise **Pivoine & Lilas**, artisan fleuriste gérée par la mère de Mattieu Mourroux. Ce projet est né d'une démarche solidaire : aider la mère d'un ami et co-développeur à développer son activité en lui concevant une boutique en ligne professionnelle. Il s'agit de la conception et du développement d'une application e-commerce complète permettant à l'entreprise de vendre ses produits en ligne.

L activité de Pivoine & Lilas comprend : - La vente de fleurs fraîches et compositions florales - La vente de vases, parfums, plantes d'intérieur, accessoires et coffrets cadeaux - Un service d'abonnement floral (livraisons régulières) - Des prestations événementielles (mariages, événements d'entreprise) - Des services B2B pour les entreprises

Cahier des charges / Expression des besoins

Objectif principal : Développer une application web e-commerce full-stack permettant à un artisan fleuriste de vendre ses produits en ligne.

Besoins fonctionnels :

1. **Catalogue produits** : Affichage des produits organisés par catégories (Fleurs Fraîches, Vases, Parfums, Plantes d'Intérieur, Accessoires, Compositions, Cadeaux), avec filtres, recherche par nom, et tri par prix
2. **Panier d'achat** : Ajout/suppression de produits, modification des quantites, calcul automatique du total
3. **Paiement en ligne** : Intégration Stripe pour le paiement sécurisé par carte bancaire (EUR)
4. **Gestion des comptes** : Inscription, connexion, déconnexion, suppression de compte
5. **Tableau d'administration** : Interface CRUD pour gérer les produits, catégories et utilisateurs (accessible uniquement aux administrateurs)

6. **API REST** : Endpoints documentés via Swagger pour toutes les opérations CRUD
7. **Pages vitrine** : Accueil, portfolio, événementiel, entreprises, contact
8. **Abonnement floral** : Page de souscription à un abonnement de livraisons régulières

Besoins non-fonctionnels :

- Interface responsive (desktop, tablette, mobile)
- Sécurité : hashage des mots de passe, authentification JWT, validation des entrées
- Performance : chargement rapide des pages, requêtes optimisées
- Maintenabilite : architecture MVC, pattern repository, code documenté

Contraintes du projet et livrables attendus

Contraintes :

- Délai : projet a réaliser dans le temps imparti de la formation Holberton School
- Technologies retenues : Python/Flask pour le back-end, HTML/CSS/JS pour le front-end
- Base de données relationnelle obligatoire (PostgreSQL)
- Paiement en ligne fonctionnel (API Stripe en mode test)
- Application déployable en local

Livrables attendus :

- Code source complet (dépôt Git)
- Application fonctionnelle avec toutes les pages et fonctionnalités
- Base de données initialisée avec données de demonstration (7 catégories, 10 produits, 1 admin)
- Documentation technique (Swagger UI, README)
- Présentation orale du projet

Environnement humain, technique et objectifs de qualité

Équipe / collaborateurs :

- Développeur : Jaille Dimitri
- Développeur : Mattieu Mourroux
- Commanditaire : Pivoine & Lilas (artisan fleuriste)
- Encadrement : Équipe pédagogique Holberton School

Environnement technique (stack, outils, méthodes) :

Catégorie	Technologies
Langages front-end	HTML5, CSS3, JavaScript (ES6+)
Frameworks front-end	Jinja2 (moteur de templates), Stripe.js v3
Langages back-end	Python 3.10
Frameworks back-end	Flask 3.1.0, Flask-RESTX 1.3.0, Flask-SQLAlchemy 3.1.1, Flask-Migrate 4.1.0

Catégorie	Technologies
Base de données	PostgreSQL (via psycopg2), SQLAlchemy 2.0.23 (ORM)
Outils de versioning	Git, GitHub
Outils de déploiement	Flask development server, python-dotenv pour la configuration
Outils de test	coverage 7.6.12, scripts de test BDD (test_db.py, test_db.sh)
Autres	Stripe API (paiement), PyJWT (authentification), Werkzeug (hashage), Google reCAPTCHA, Flask-CORS, Swagger/OpenAPI 3.0

Objectifs de qualité :

- Code structuré selon le pattern MVC (Models / Views-Templates / Controllers-Routes)
- Pattern Repository pour l'accès aux données (séparation des responsabilités)
- Facade de services pour centraliser la logique métier
- API RESTful documentée automatiquement via Swagger UI
- Gestion des erreurs avec try/catch et rollback des transactions
- Variables sensibles externalisées dans des fichiers .env

Méthodologie de gestion de projet :

- Méthode itérative : développement incremental des fonctionnalités
- Versioning Git avec commits réguliers
- Gestion des migrations de base de données avec Alembic (Flask-Migrate)
- 6 migrations successives traçant l'évolution du schema (ajout champ password étendu, ajout promotions, ajout is_on_sale, suppression product_id des reviews)

Éléments significatifs côté front-end

Maquettes de l'application (responsive)

L'application comprend 16 pages HTML, chacune maquettée selon la charte graphique de Pivoine & Lilas :

Page	Fichier	Description
Accueil	accueil.html	Page d'accueil avec présentation de la marque
Boutique	shop.html (461 lignes)	Catalogue produits avec filtres, recherche, panier

Page	Fichier	Description
Panier	panier.html (375 lignes)	Gestion du panier avec quantites modifiables, prix unitaire et total
Paiement	checkout.html (588 lignes)	Formulaire de paiement Stripe avec résumé commande
Paiement alternatif	payment.html (406 lignes)	Page de paiement alternative
Abonnement	subscription.html (182 lignes)	Page d'abonnement floral
Paiement abonnement	subscription_payment.html (593 lignes)	Paiement récurrent Stripe
Administration	admin.html (748 lignes)	Dashboard CRUD (catégories, produits, utilisateurs)
Connexion	account.html (149 lignes)	Formulaire de connexion
Inscription	register.html (127 lignes)	Formulaire d'inscription avec reCAPTCHA
Contact	contact.html (114 lignes)	Page de contact
Entreprises	entreprises.html (114 lignes)	Services B2B
Événementiel	événementiel.html (112 lignes)	Services événementiels
Portfolio	portfolio.html (138 lignes)	Galerie de realisations
Mot de passe oublie	forgot-password.html (83 lignes)	Récupération de mot de passe
Vérification code	verify-code.html (34 lignes)	Saisie du code de vérification

Schema d'enchainement des maquettes

```
Accueil ──> Boutique ──> Panier ──> Checkout ──> Paiement Stripe
   │            │                                      │
   │            └──> Detail produit                    └──> Confirmation
   │
   ├──> Evenementiel
   ├──> Entreprises
   ├──> Portfolio
   ├──> Contact
   ├──> Abonnement ──> Paiement abonnement
   │
   ├──> Connexion ──> Profil (deconnexion, suppression)
   ├──> Inscription (reCAPTCHA)
```

```
├──> Mot de passe oublie ──> Verification code
│
└──> Administration (si admin)
         ├──> CRUD Categories
         ├──> CRUD Produits
         └──> CRUD Utilisateurs
```

Navigation commune à toutes les pages via la barre de navigation fixe : Accueil | Shop | Événementiel | Entreprises | Portfolio | Contact | Abonnement | Panier | Admin (si admin) | Profil

Captures d'écran de l'interface utilisateur (responsive)

Les captures d'écran ci-dessous présentent les pages principales de l'application en version desktop (1280x800) et mobile (393x852, simulation iPhone).

Page d'accueil — Présentation de la marque Pivoine & Lilas avec navigation principale.

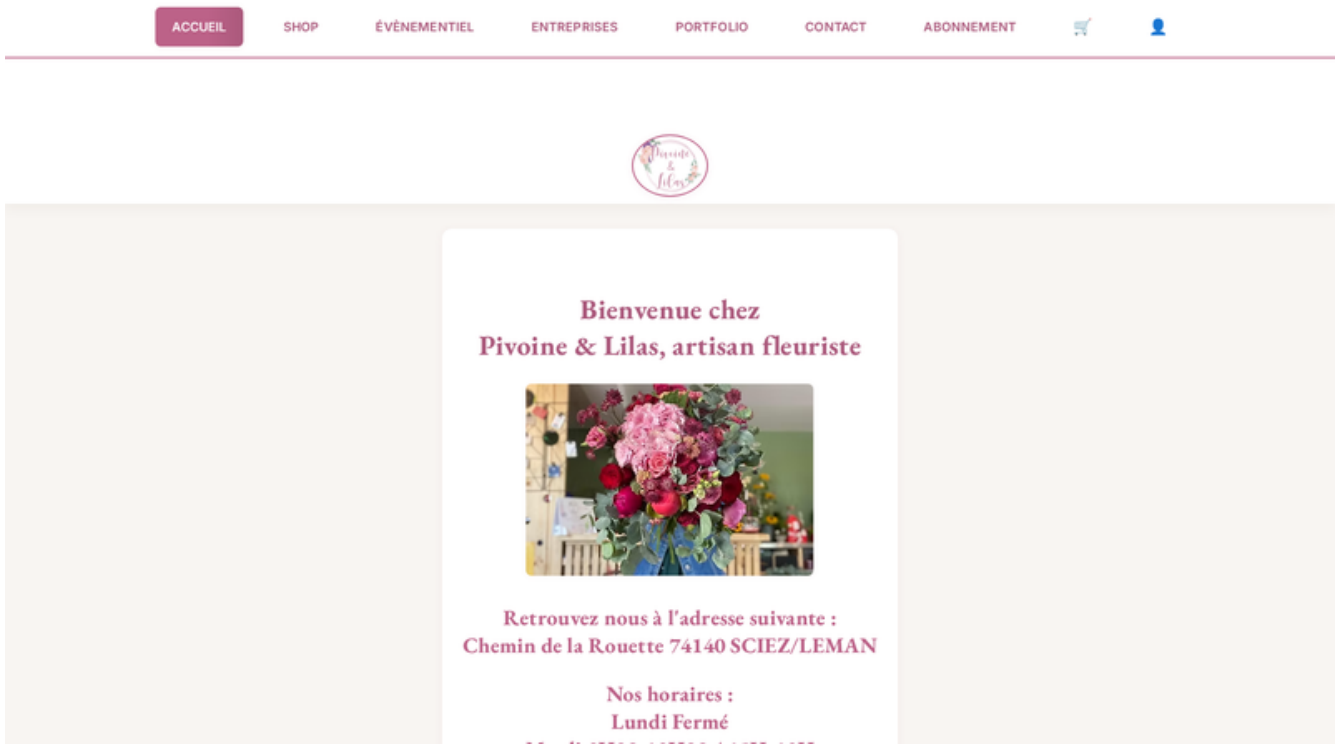


Fig. 3 : Page d'accueil - Desktop

Fig. 4 : Page d'accueil - Mobile

Page boutique — Grille de produits avec filtres (catégories, prix, recherche).

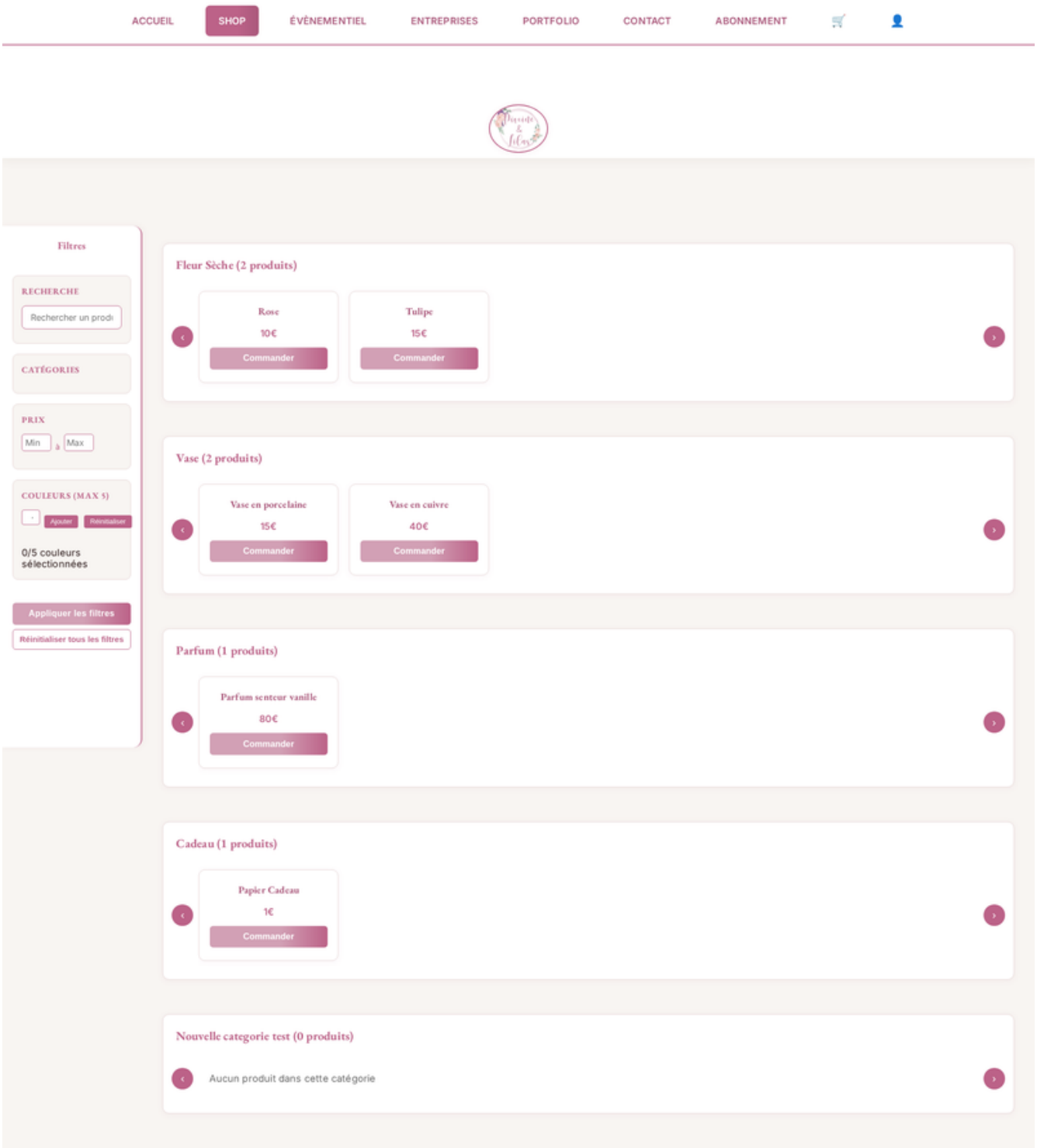


Fig. 5 : Boutique - Desktop

Fig. 6 : Boutique - Mobile

Page panier — Articles avec quantites modifiables, prix unitaire et total.

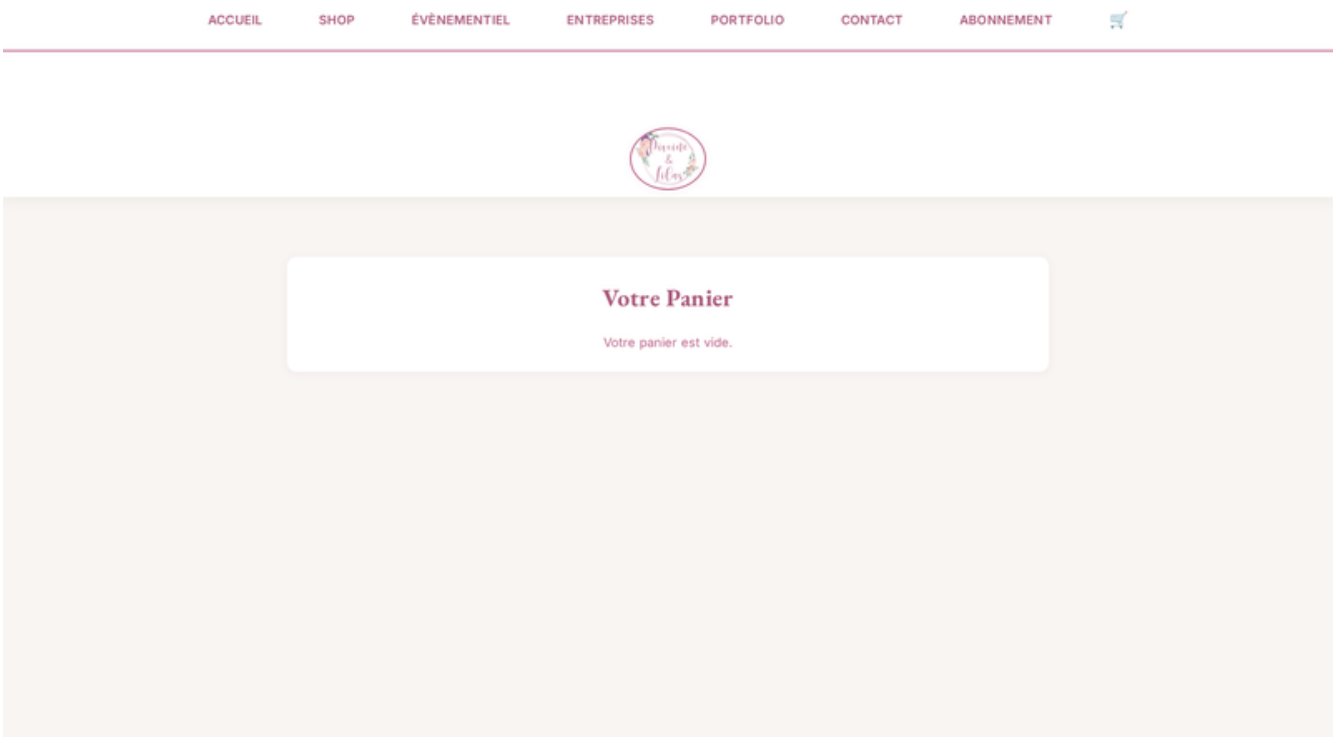


Fig. 7 : Panier - Desktop

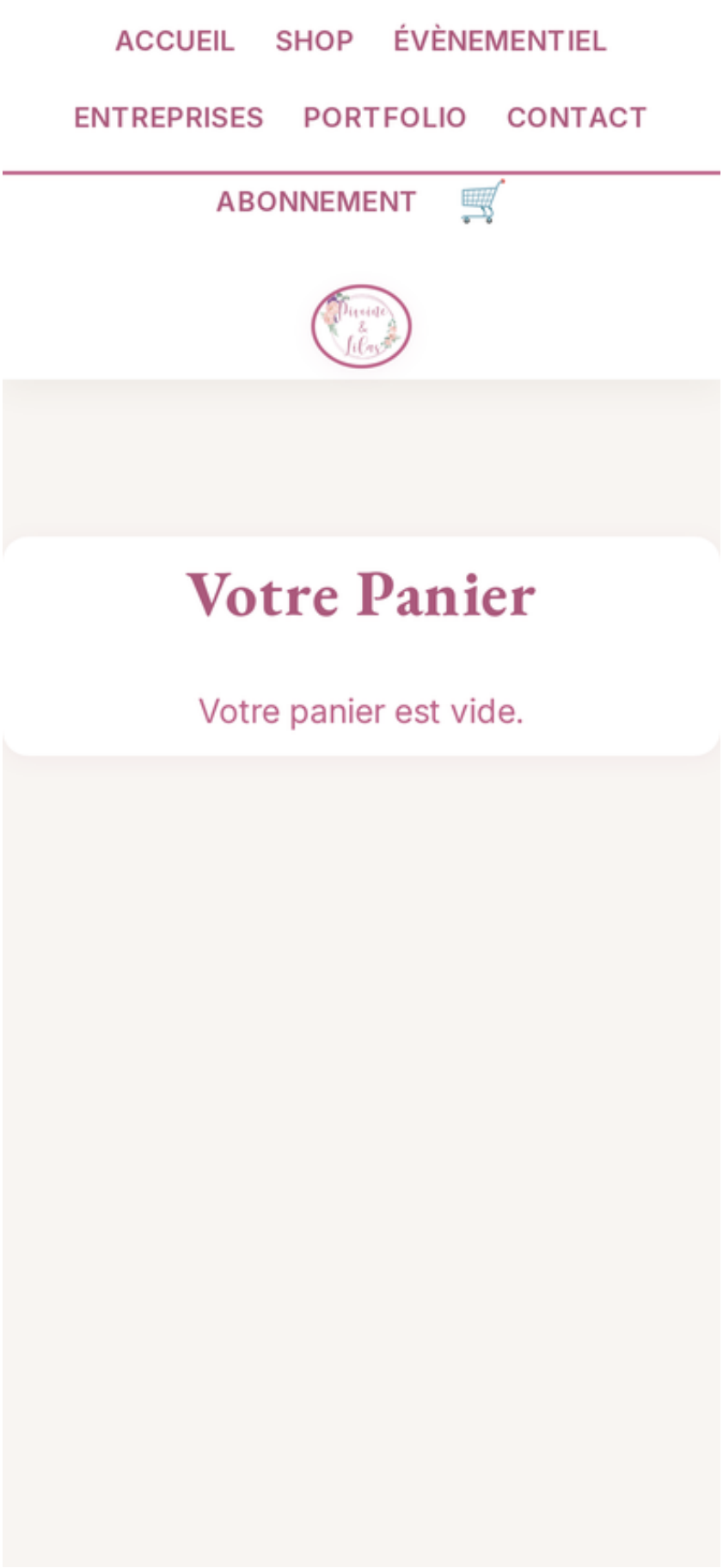


Fig. 8 : Panier - Mobile

Page de paiement Stripe — Formulaire Stripe Éléments, résumé de commande.

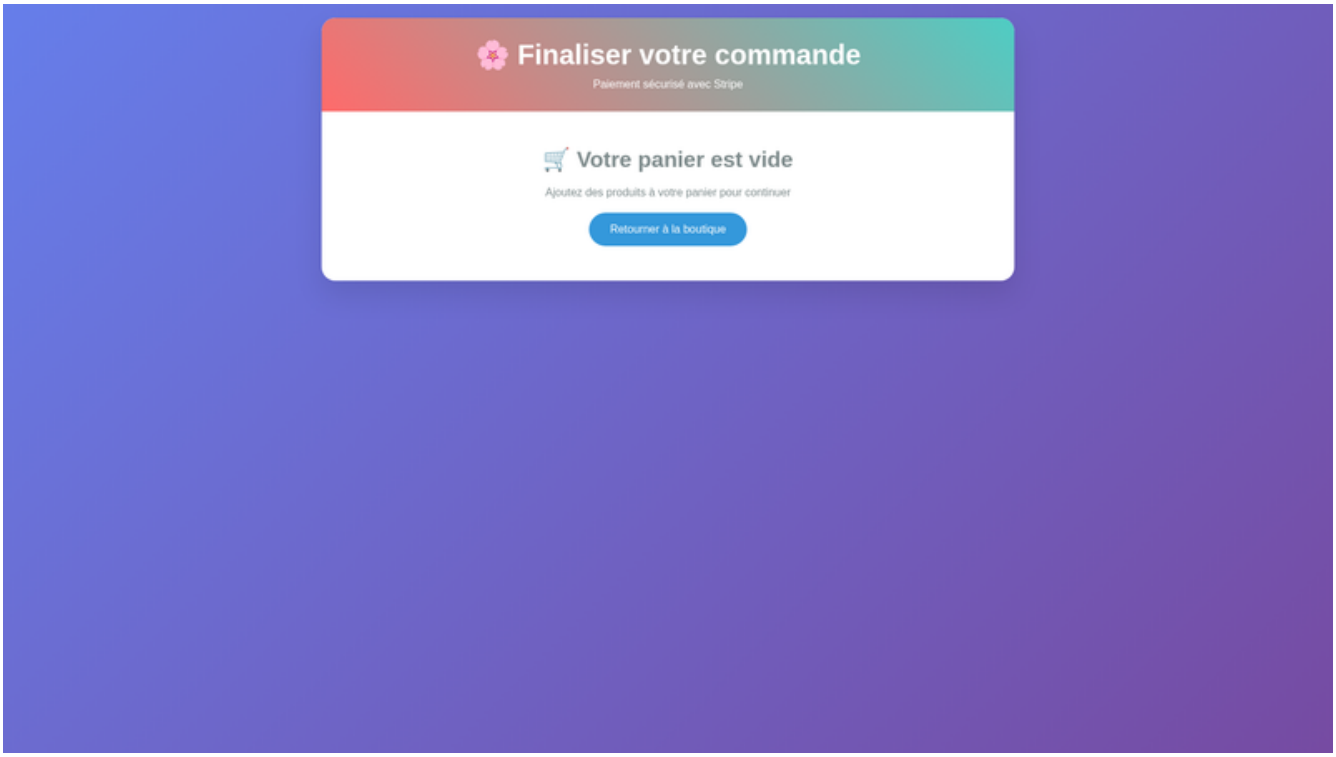


Fig. 9 : Checkout - Desktop

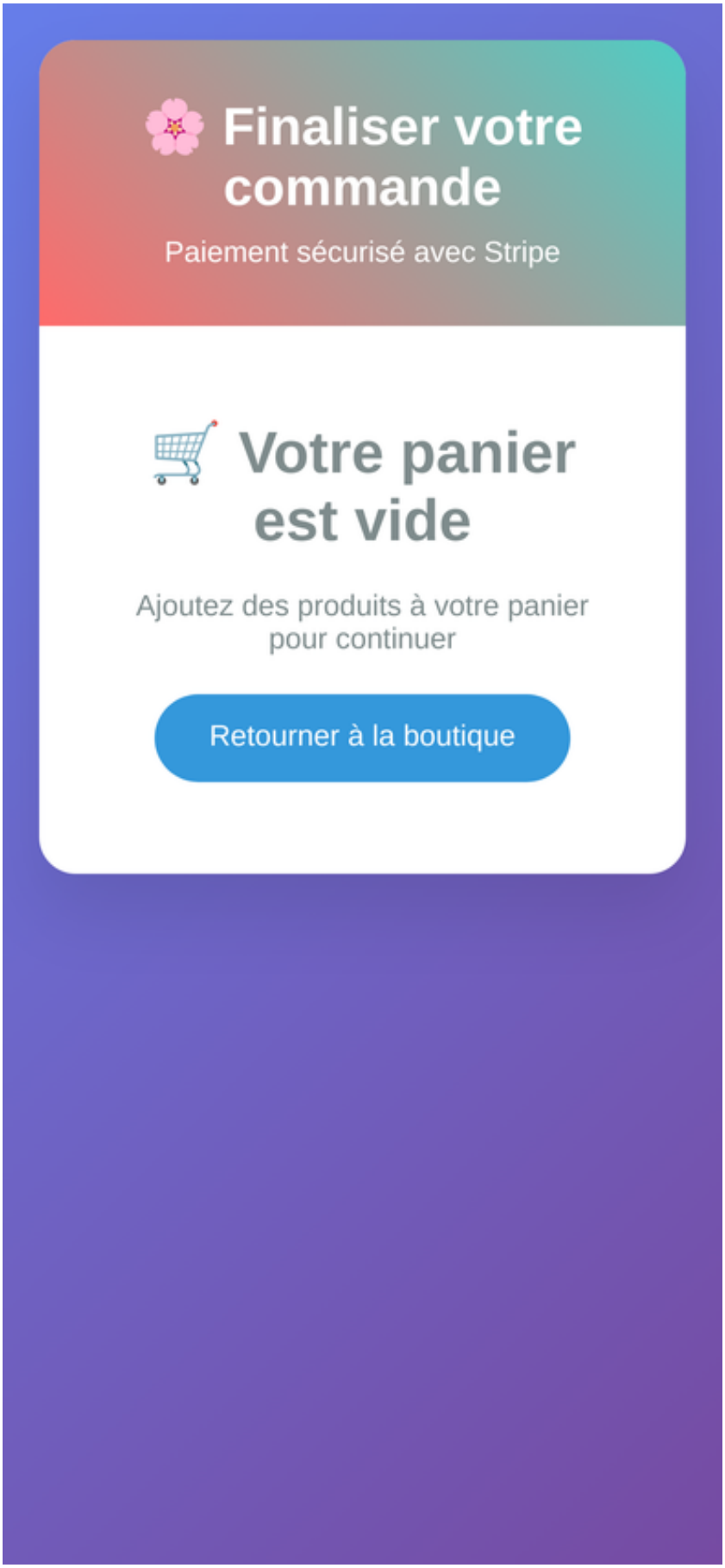


Fig. 10 : Checkout - Mobile

Documentation API Swagger UI — Endpoints classés par namespace (Auth, Products, Catégories, Users, Payments).

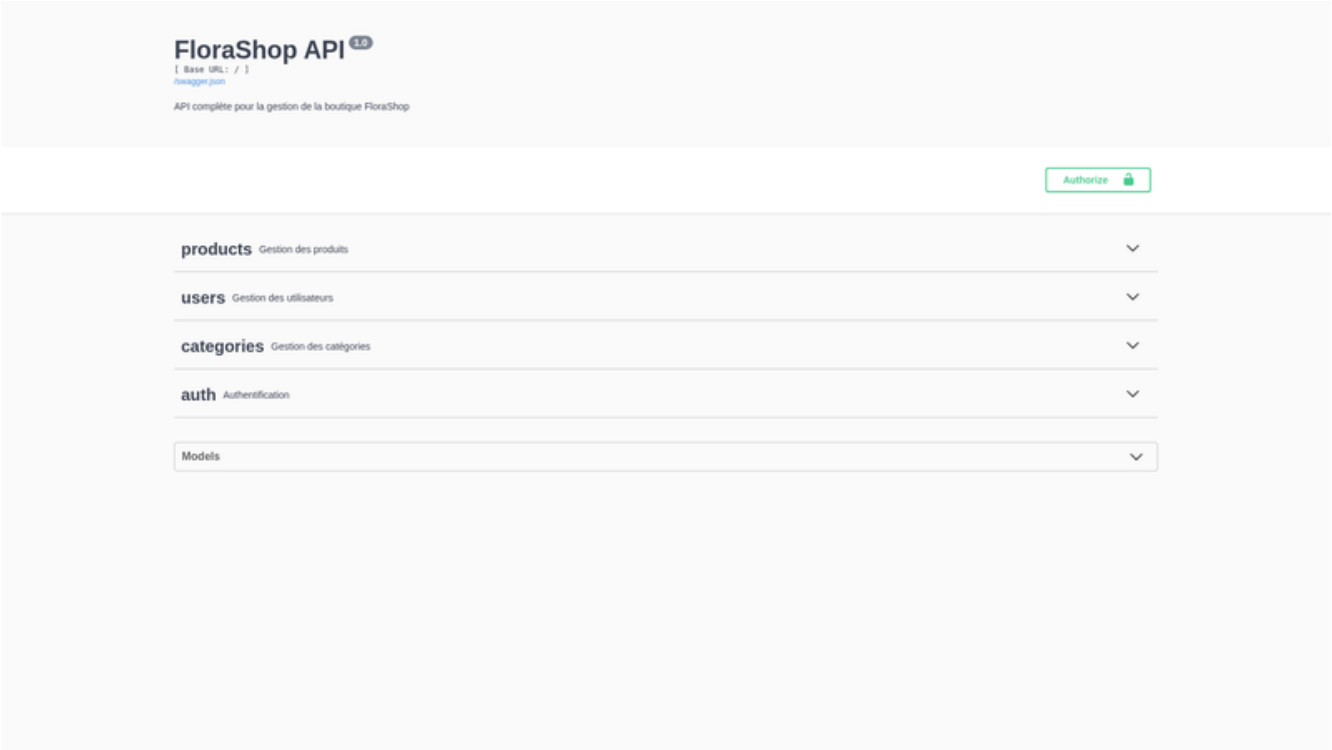


Fig. 11 : Swagger UI - Desktop

Argumentation responsive :

- Les captures montrent le même contenu adapté a 2 resolutions (desktop 1280px et mobile 393px)
- La navigation s'adapté avec des espacements et tailles de police réduites sur mobile (`gap: 7px`, `font-size: 15px`)
- La grille de produits passe de 3 colonnes a 1 colonne sur mobile (media query `@media max-width: 600px`)
- Le formulaire de paiement Stripe s'adapté automatiquement à la largeur de l'écran
- Le panier conserve toutes les fonctionnalités en version mobile (modification quantite, suppression)

Extraits de code d'interface utilisateur statique

```
<!-- shop.html — Structure de la page boutique avec navigation responsive -->
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Boutique Florale</title>
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```html
  <link rel="stylesheet" href="../static/css/style.css">
</head>
<body>
  <header>
    <img class="logo" src="..." alt="Logo pivoine & lilas, artisan fleuriste">
    <nav>
      <ul>
        <li><a href="accueil.html">ACCUEIL</a></li>
        <li><a href="shop.html" class="active">SHOP</a></li>
        <li><a href="evenementiel.html">EVENEMENTIEL</a></li>
        <li><a href="entreprises.html">ENTREPRISES</a></li>
        <li><a href="portfolio.html">PORTFOLIO</a></li>
        <li><a href="contact.html">CONTACT</a></li>
        <li><a href="subscription.html">ABONNEMENT</a></li>
        <li><a href="panier.html" id="cart-link">Panier
          <span id="cart-count">0</span></a>
        </li>
        <li id="admin-access" style="display:none;">
          <a href="admin.html">Admin</a>
        </li>
      </ul>
    </nav>
  </header>
  <div class="shop-container">
    <aside class="sidebar-filters">
      <h2>Filtres</h2>
      <input type="text" placeholder="Rechercher un produit..." id="searchInput">
      <div id="categories-filters">
        <!-- Categories chargees dynamiquement depuis l API -->
      </div>
      <input type="number" placeholder="Min" id="minPrice">
      <input type="number" placeholder="Max" id="maxPrice">
    </aside>
    <main class="product-grid" id="productGrid">
      <!-- Produits charges dynamiquement -->
    </main>
  </div>
</body>
</html>
```

```css
/* style.css — Variables CSS et design vintage de la marque Pivoine & Lilas */
@import
↪  url('https://fonts.googleapis.com/css2?family=Inter:wght@400;600&family=EB+Garamond:wght@400;700&di

:root {
    --vintage-bg: #fff6f2;
    --vintage-main: #bc6288;
    --vintage-main-dark: #ab597c;
    --vintage-accent: #d2a0b5;
    --vintage-text: #231111;
    --vintage-card: #fff;
    --vintage-border: #e7d6d6;
    --vintage-shadow: 0 2px 12px rgba(188,98,136,0.07);
}
```

```
body {
    background: var(--vintage-bg);
    color: var(--vintage-text);
    font-family: 'EB Garamond', 'Inter', serif;
    font-size: 18px;
}

nav {
    position: fixed;
    top: 0;
    width: 100%;
    z-index: 1000;
    background: var(--vintage-card);
    border-bottom: 2px solid var(--vintage-main);
    height: 90px;
    display: flex;
    align-items: center;
    justify-content: center;
}

nav a {
    color: var(--vintage-main-dark);
    font-family: 'EB Garamond', serif;
    font-weight: 600;
    font-size: 19px;
    padding: 13px 24px;
    border-radius: 8px;
    transition: background 0.18s, color 0.18s;
}
```

Argumentation :

- **Charte graphique cohérente** : utilisation de variables CSS (`:root`) pour garantir l'uniformite des couleurs sur toutes les pages. La palette vintage (#bc6288 rose, #fff6f2 fond creme) correspond à l'univers fleuri de la marque.
- **Responsive design** : balise `<meta name="viewport">` présente sur toutes les pages, navigation fixe adaptée aux differentes tailles d'écran.
- **Accessibilite** : attributs `alt` sur les images, contrastes de couleurs suffisants (texte sombre #231111 sur fond clair #fff6f2).
- **Separation des responsabilités** : HTML semantique (header, nav, aside, main) séparé de la mise en forme CSS dans des fichiers dédiés (style.css, account.css, subscription.css).
- **Polices web** : import Google Fonts (EB Garamond pour le style éditorial, Inter pour la lisibilité).

CSS responsive — media queries

```
/* style.css — Media queries pour l adaptation tablette et mobile */
@media (max-width: 900px) {
    .shop-container {
        flex-direction: column;  /* Sidebar passe au-dessus des produits */
```

```
        gap: 0;
    }
    .sidebar-filters {
        position: static;        /* Plus de sidebar fixe */
        width: 100%;
        height: auto;
        border-radius: 0 0 12px 12px;
        box-shadow: none;
        border-right: none;
        margin-bottom: 2rem;
    }
    .products-area {
        padding: 1rem 2vw;
    }
    .products-grid {
        grid-template-columns: 1fr;  /* 1 produit par ligne sur tablette */
        gap: 1.2rem;
    }
}

@media (max-width: 600px) {
    .main-content {
        padding: 8px 2vw 14px 2vw;  /* Marges reduites sur mobile */
    }
    nav ul {
        gap: 7px;                    /* Navigation compacte */
    }
    nav a {
        padding: 8px 7px;
        font-size: 15px;             /* Taille reduite pour petit ecran */
    }
    .logo {
        width: 55px;
        height: 40px;
        margin: 60px auto 6px auto;  /* Logo redimensionne */
    }
}
```

Argumentation responsive :

- **Mobile-first implicite** : les media queries ciblent les breakpoints 900px (tablette) et 600px (mobile). En dessous de 900px, le layout passe de 2 colonnes (sidebar + grille) a 1 colonne empilée.
- **Grille CSS adaptative** : `grid-template-columns: repeat(auto-fill, minmax(260px, 1fr))` sur desktop permet un nombre variable de colonnes selon la largeur, puis se réduit a `1fr` (1 colonne) sur mobile.
- **Navigation responsive** : espacement et taille de police reduits sur mobile (`gap: 7px`, `font-size: 15px`) pour que tous les liens restent accessibles sans débordement.

Formulaire sécurisé — inscription avec reCAPTCHA

```html
<!-- register.html — Formulaire d inscription avec validation et protection anti-bot -->
<form class="login-form" id="register-form">
    <h1>Creation de compte</h1>
    <div class="form-group">
        <label for="username">Nom d utilisateur</label>
        <input type="text" id="username" required>
    </div>
    <div class="form-group">
        <label for="email">Email</label>
        <input type="email" id="email" required>  <!-- Validation HTML5 du format email -->
    </div>
    <div class="form-group">
        <label for="password">Mot de passe</label>
        <input type="password" id="password" required>
    </div>
    <div class="form-group">
        <label for="confirm-password">Confirmer le mot de passe</label>
        <input type="password" id="confirm-password" required>
    </div>
    <div class="form-group">
        <!-- Protection anti-bot Google reCAPTCHA -->
        <div class="g-recaptcha" data-sitekey="YOUR_SITE_KEY"></div>
    </div>
    <button type="submit" class="login-button">Creer mon compte</button>
</form>

<script>
    document.getElementById('register-form').addEventListener('submit', async function(e) {
        e.preventDefault();
        const password = document.getElementById('password').value;
        const confirmPassword = document.getElementById('confirm-password').value;
        // Validation cote client : verification de correspondance des mots de passe
        if (password !== confirmPassword) {
            alert('Les mots de passe ne correspondent pas');
            return;
        }
        const result = await apiService.register(
            document.getElementById('username').value,
            document.getElementById('email').value,
            password
        );
        if (result.success) {
            alert('Compte cree avec succes !');
            window.location.href = 'accueil.html';
        } else {
            alert(result.error || 'Erreur lors de la creation du compte');
        }
    });
</script>
```

Argumentation sécurité front-end :

- **Validation HTML5 native** : les attributs `type="email"` et `required` empêchent la soumission de formulaires avec des champs vides ou un format d'email invalide, avant

même que le JavaScript né s'exécuté.

- **Confirmation de mot de passe** : la vérification `password !== confirmPassword` côté client empêche les erreurs de saisie.
- **reCAPTCHA** : le widget Google reCAPTCHA protégé le formulaire contre les inscriptions automatisées (bots). La clé publique est chargée via `<script src="https://www.google.com/re` avec les attributs `async defer` pour né pas bloquer le rendu de la page.
- **Échappement Jinja2** : sur toutes les pages, les variables affichees dans les templates Jinja2 sont automatiquement échappées par le moteur de templates, protégeant contre les attaques XSS (Cross-Site Scripting).

Extraits de code de la partie dynamique des interfaces utilisateur

```
// api.js — Classe ApiService gerant les appels API et l authentification cote client
const API_BASE_URL = 'http://localhost:5000/api/v1';

class ApiService {
    constructor() {
        this.token = localStorage.getItem('auth_token');
        this.user = JSON.parse(localStorage.getItem('user') || 'null');
        this.updateProfileUI();
    }

    getHeaders() {
        const headers = { 'Content-Type': 'application/json' };
        if (this.token) {
            headers['Authorization'] = `Bearer ${this.token}`;
        }
        return headers;
    }

    async login(email, password) {
        const response = await fetch(`${API_BASE_URL}/auth/login`, {
            method: 'POST',
            headers: { 'Content-Type': 'application/json', 'Accept': 'application/json' },
            body: JSON.stringify({ email, password })
        });
        const data = await response.json();
        if (response.ok && data.success) {
            this.user = data.data.user;
            this.token = data.data.token;
            localStorage.setItem('auth_token', this.token);
            localStorage.setItem('user', JSON.stringify(this.user));
            this.updateProfileUI();
            return { success: true, data: data.data };
        }
        return { success: false, error: data.message || 'Erreur de connexion' };
    }

    async register(username, email, password) {
        const response = await fetch(`${API_BASE_URL}/auth/register`, {
            method: 'POST',
            headers: { 'Content-Type': 'application/json', 'Accept': 'application/json' },
            body: JSON.stringify({ username, email, password })
```

```
        });
        const data = await response.json();
        if (response.ok && data.success) {
            this.user = data.data.user;
            localStorage.setItem('user', JSON.stringify(this.user));
            this.updateProfileUI();
            return { success: true, data: data.data };
        }
        return { success: false, error: data.message || 'Erreur lors de la creation du
        ↪  compte' };
    }

    logout() {
        this.token = null;
        this.user = null;
        localStorage.removeItem('auth_token');
        localStorage.removeItem('user');
        this.updateProfileUI();
    }

    // Gestion des produits — appels CRUD vers l API
    async getProducts() {
        const response = await fetch(`${API_BASE_URL}/products`, { headers: this.getHeaders()
        ↪  });
        return await response.json();
    }

    async getCategories() {
        const response = await fetch(`${API_BASE_URL}/categories`, { headers:
        ↪  this.getHeaders() });
        return await response.json();
    }
}
```

Argumentation :

- **Architecture orientée objet** : la classé `ApiService` encapsule toute la logique d'appel API, la gestion du token JWT et la mise à jour de l'interface utilisateur.
- **Authentification côté client** : le token JWT est stocké dans le `localStorage` et envoyé automatiquement dans le header `Authorization: Bearer` de chaque requête.
- **Gestion de session** : les méthodes `login()`, `register()`, `logout()` gèrent l'etat de l'utilisateur et mettent à jour l'interface (affichage du bouton connexion/déconnexion, email utilisateur, accès admin).
- **Appels API asynchrones** : utilisation de `async/await` avec `fetch()` pour des appels non-bloquants et une gestion propre des erreurs.
- **Separation front-end / back-end** : le front-end communique exclusivement via l'API REST, ce qui garantit une architecture découplable.

Gestion dynamique du panier (JavaScript)

```
// panier.html — Fonctions de gestion du panier cote client
function renderCart() {
    const cart = JSON.parse(localStorage.getItem('cart') || '[]');
    const cartContent = document.getElementById('cart-content');
    const cartSummary = document.getElementById('cart-summary');
    const cartTotal = document.getElementById('cart-total');

    if (cart.length === 0) {
        cartContent.innerHTML = "<p>Votre panier est vide.</p>";
        cartSummary.style.display = "none";
        return;
    }

    let total = 0;
    cartContent.innerHTML = `
        <table class="cart-table">
          <thead>
            <tr><th>Produit</th><th>Categorie</th><th>Prix</th><th>Retirer</th></tr>
          </thead>
          <tbody>
            ${cart.map((prod, idx) => {
              total += Number(prod.price);
              return `
                <tr>
                  <td>${prod.name}</td>
                  <td>${prod.category}</td>
                  <td>${Number(prod.price).toFixed(2)} EUR</td>
                  <td><button class="remove-btn"
↪   onclick="removeFromCart(${idx})">X</button></td>
                </tr>`;
            }).join('')}
          </tbody>
        </table>`;
    cartTotal.textContent = "Total : " + total.toFixed(2) + " EUR";
    cartSummary.style.display = "flex";
}

function removeFromCart(idx) {
    let cart = JSON.parse(localStorage.getItem('cart') || '[]');
    cart.splice(idx, 1);
    localStorage.setItem('cart', JSON.stringify(cart));
    renderCart();
    updateCartCount();
}

function updateCartCount() {
    const cart = JSON.parse(localStorage.getItem('cart') || '[]');
    const badge = document.getElementById('cart-count');
    if (badge) {
        badge.textContent = cart.length;
        badge.style.display = cart.length > 0 ? 'inline-block' : 'none';
    }
}
```

Argumentation :

- **Persistance locale** : le panier est stocké dans le `localStorage` du navigateur sous forme de tableau JSON. L'utilisateur conserve son panier même après fermeture du navigateur (sans cookies serveur).
- **Rendu dynamique du DOM** : la fonction `renderCart()` reconstruit le tableau HTML à chaque modification, utilisant `Array.map()` et les template literals ES6 pour générer les lignes du tableau.
- **Mise à jour en temps reel** : le compteur du badge panier (`cart-count`) est mis à jour à chaque ajout ou suppression via `updateCartCount()`.
- **Manipulation sécurisée** : `splice(idx, 1)` supprimé l'élément à l'index exact sans affecter les autres éléments du panier.

Intégration du paiement Stripe.js (checkout)

```
// checkout.html — Integration Stripe.js v3 pour le paiement securise
let stripe = null;
let elements = null;
let card = null;

async function initializeStripe() {
    // Appel API pour creer un Payment Intent et obtenir la cle publique Stripe
    const response = await fetch('/api/v1/payments/create-payment-intent', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
            items: cartItems.map(item => ({
                product_id: item.product_id,
                quantity: item.quantity
            })),
            email: 'temp@example.com'
        })
    });

    if (response.ok) {
        const data = await response.json();
        // Initialisation de Stripe avec la cle publique retournee par le serveur
        stripe = Stripe(data.stripe_publishable_key);
        elements = stripe.elements();
        card = elements.create('card', {
            style: {
                base: { fontSize: '16px', color: '#424770',
                        '::placeholder': { color: '#aab7c4' } }
            }
        });
        card.mount('#card-element');

        // Affichage des erreurs de saisie en temps reel
        card.on('change', ({error}) => {
            const displayError = document.getElementById('card-errors');
            displayError.textContent = error ? error.message : '';
            displayError.style.display = error ? 'block' : 'none';
        });
    }
```

```
}

// Soumission du formulaire de paiement
document.getElementById('payment-form').addEventListener('submit', async (event) => {
    event.preventDefault();
    const email = document.getElementById('email').value;
    const name = document.getElementById('name').value;

    // Creer le Payment Intent definitif
    const response = await fetch('/api/v1/payments/create-payment-intent', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
            items: cartItems.map(item => ({
                product_id: item.product_id, quantity: item.quantity
            })),
            email: email
        })
    });
    const { client_secret, order_id } = await response.json();

    // Confirmation du paiement via Stripe.js
    const {error, paymentIntent} = await stripe.confirmCardPayment(client_secret, {
        payment_method: {
            card: card,
            billing_details: { name: name, email: email }
        }
    });

    if (error) {
        // Affichage de l erreur de paiement
        document.getElementById('payment-result').innerHTML =
            `<div class="error"><h3>Erreur de paiement</h3><p>${error.message}</p></div>`;
    } else {
        // Paiement reussi : affichage de la confirmation
        document.getElementById('payment-result').innerHTML =
            `<div class="success"><h3>Paiement reussi !</h3>
             <p>Commande #${order_id} confirmee</p>
             <p>Transaction: ${paymentIntent.id}</p></div>`;
        localStorage.removeItem('cart');  // Vider le panier
    }
});
```

Argumentation sécurité paiement :

- **Délégation a Stripe.js** : les coordonnees bancaires (numero de carte, date, CVC) né transitent jamais par notre serveur. Le composant `card` de Stripe Éléments les envoie directement aux serveurs Stripe via un iframe sécurisé (conformité PCI DSS).
- **Flux Payment Intent** : le `client_secret` est généré côté serveur avec un montant recalculé à partir des prix en base (pas de montant venant du client). Stripe confirmé le paiement via ce secret unique.
- **Validation en temps reel** : `card.on('change')` affiche les erreurs de saisie de carte instantanément, sans requête serveur.

- **Nettoyage post-paiement** : après un paiement réussi, le panier est vide (`localStorage.removeItem('cart')`) et le formulaire est masqué pour empêcher un double paiement.
- **Gestion des erreurs** : les erreurs Stripe (carte refusée, fonds insuffisants…) sont affichees à l'utilisateur sans exposer de details techniques sensibles.

Éléments significatifs côté back-end

Architecture technique de l'application

```
┌──────────────────────────────────────────────────────────────────────┐
│                        NAVIGATEUR CLIENT                             │
│  HTML5 / CSS3 / JavaScript (api.js, auth.js) / Stripe.js v3          │
└──────────────────────────────────────────────────────────────────────┘
                              │ HTTP / HTTPS (JSON)
                              ▼
┌──────────────────────────────────────────────────────────────────────┐
│                        SERVEUR FLASK (Python 3.10)                   │
│                                                                      │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────────────┐       │
│  │ Flask-RESTX  │  │   Jinja2     │  │   Flask-Migrate      │       │
│  │ (API REST)   │  │ (Templates)  │  │   (Alembic)          │       │
│  └──────────────┘  └──────────────┘  └──────────────────────┘       │
│         │                  │                                         │
│  ┌──────────────────────────────────────────────────────────┐       │
│  │                Services metier                           │       │
│  │ StripeService  │  Facade  │  Auth (PyJWT)                │       │
│  └──────────────────────────────────────────────────────────┘       │
│         │                  │                                         │
│  ┌──────────────────────────────────────────────────────────┐       │
│  │              Repositories (Pattern Repository)            │       │
│  │ CategoryRepo │ ProductRepo │ UserRepo │ PriceRepo         │       │
│  └──────────────────────────────────────────────────────────┘       │
│                              │                                       │
│  ┌──────────────────────────────────────────────────────────┐       │
│  │             SQLAlchemy ORM (Models)                       │       │
│  │ User │ Product │ Category │ Order │ OrderItem │ ... │     │       │
│  └──────────────────────────────────────────────────────────┘       │
└──────────────────────────────────────────────────────────────────────┘
                              │ SQL (psycopg2)
                              ▼
┌──────────────────────────────────────────────────────────────────────┐
│                POSTGRESQL (base florashop)                           │
│  7 tables : users, categories, products, orders,                     │
```

```
│             order_items, reviews, prices                          │
└──────────────────────────────────────────────────────────────────┘

          Integrations externes :
          ┌──────────────────────────┐
          │     API Stripe           │
          │  Payment Intents         │
          │  Webhooks (signatures)   │
          └──────────────────────────┘
```

Argumentation :

- **Architecture 3-tiers** : la séparation entre le navigateur (présentation), le serveur Flask (logique métier) et PostgreSQL (données) garantit la maintenabilité et la scalabilité.
- **Couches internes du serveur** : les routes (Flask-RESTX) délèguent aux services métier, qui délèguent aux repositories, qui utilisent l'ORM SQLAlchemy. Cette architecture en couches facilite les tests et la réutilisation du code.
- **Intégration externe** : l'API Stripe est appelée uniquement depuis le `StripeService` côté serveur (jamais directement depuis le client), sauf Stripe.js qui communique directement avec Stripe pour les données bancaires (conformité PCI DSS).

Présentation de la base de données

Schema conceptuel (MCD) - données et relations

Le MCD suit la méthode Merise. Les 7 entités du système et leurs associations sont représentées ci-dessous.

Entités :

Entité	Attributs	Identifiant
USERS	username (VARCHAR 50, unique, NN), email (VARCHAR 120, unique, NN), password (VARCHAR 255, NN), is_admin (BOOL, défaut FALSE)	id (INT, PK)
CATEGORIES	name (VARCHAR 100, unique, NN), created_at (DATETIME)	id (INT, PK, auto)
PRODUCTS	name (VARCHAR 255, NN), price (FLOAT, NN), is_on_sale (BOOL, défaut FALSE)	id (INT, PK)

Entité	Attributs	Identifiant
ORDERS	email (VARCHAR 120, NN), total_amount (FLOAT, NN), stripe_payment_intent_id (VARCHAR 255, nullable), status (VARCHAR 50, défaut ‘pending’), created_at (DATETIME)	id (INT, PK, auto)
ORDER_ITEMS	quantity (INT, NN, défaut 1), price (FLOAT, NN — prix au moment de l’achat)	id (INT, PK, auto)
REVIEWS	content (TEXT, NN), rating (INT, NN)	id (INT, PK)
PRICES	amount (FLOAT, NN), is_active (BOOL, défaut TRUE), created_at (DATETIME), updated_at (DATETIME)	id (INT, PK)

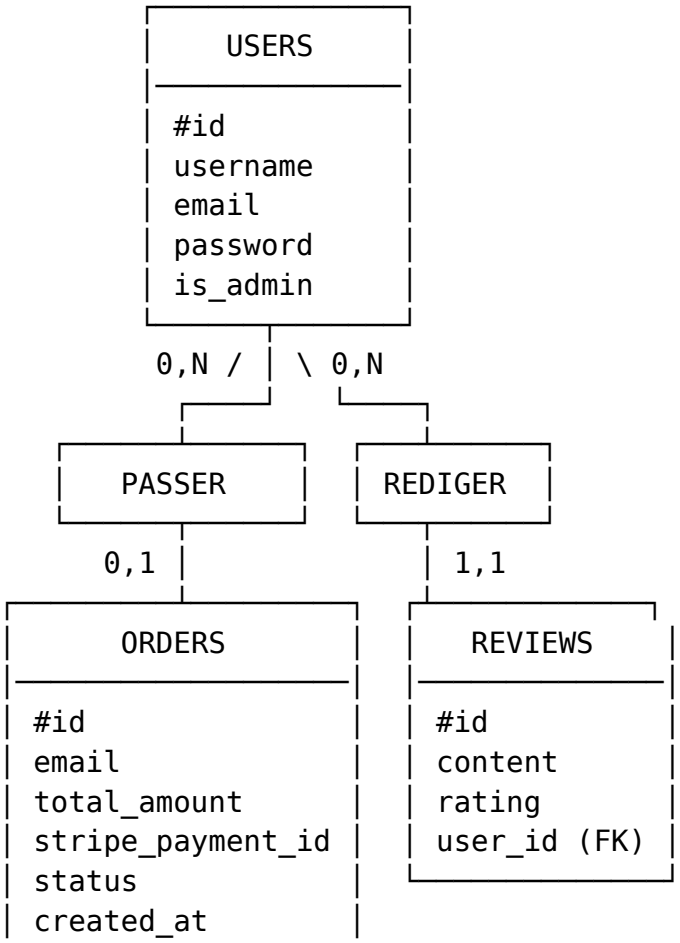
Associations et cardinalites (notation Merise) :

Association	Entité 1	Cardinalite	Entité 2	Cardinalite	Description
PASSER	USERS	0,N	ORDERS	0,1	Un utilisateur peut passer 0 a N commandes. Une commande peut être passee par un utilisateur (ou en invite, d’ou 0,1)

Association	Entité 1	Cardinalite	Entité 2	Cardinalite	Description
COMPOSER	ORDERS	1,N	ORDER_ITEMS	1,1	Une commande contient au moins 1 ligne de commande. Chaque ligne appartient à exactement 1 commande (cascade delete-orphan)
REFERENCER	ORDER_ITEMS	1,1	PRODUCTS	0,N	Chaque ligne de commande reference exactement 1 produit. Un produit peut apparaitre dans 0 a N lignes de commande
CONTENIR	CATEGORIES	1,N	PRODUCTS	1,1	Une catégorie contient 1 a N produits. Chaque produit appartient à exactement 1 catégorie (cascade delete)

Association	Entité 1	Cardinalite	Entité 2	Cardinalite	Description
REDIGER	USERS	0,N	REVIEWS	1,1	Un utilisateur peut rediger 0 a N avis. Chaque avis est rédigé par exactement 1 utilisateur
HISTORISER	PRODUCTS	0,N	PRICES	1,1	Un produit peut avoir 0 a N prix (historique). Chaque prix appartient à exactement 1 produit

Diagramme entités-relations :



```
│ user_id (FK)      │
└──────────────────┘
      1,N │
   ┌──────────────┐
   │  COMPOSER    │
   └──────────────┘
      1,1 │
┌──────────────────────────┐
│     ORDER_ITEMS          │
├──────────────────────────┤
│ #id                      │
│ quantity                 │
│ price                    │
│ order_id (FK)            │
│ product_id (FK)          │
└──────────────────────────┘
      1,1 │
   ┌──────────────┐
   │ REFERENCER   │
   └──────────────┘
      0,N │
┌──────────────────────────┐
│     PRODUCTS             │
├──────────────────────────┤
│ #id                      │
│ name                     │
│ price                    │
│ category_id (FK)         │
│ is_on_sale               │
└──────────────────────────┘
      0,N │
   ┌──────────────┐
   │  HISTORISER  │
   └──────────────┘
      1,1 │
┌──────────────────────────┐
│      PRICES              │
├──────────────────────────┤
│ #id                      │
│ amount                   │
│ is_active                │
│ product_id (FK)          │
│ created_at               │
│ updated_at               │
└──────────────────────────┘
```

```
┌──────────────────┐
│  CATEGORIES      │
├──────────────────┤
│ #id              │
│ name             │
│ created_at       │
└──────────────────┘
      1,N │
┌──────────────────┐
│    CONTENIR      │
└──────────────────┘
      1,1 │
```

Règles de gestion : - RG1 : Un utilisateur est identifié par un couple (username, email) unique - RG2 : Une commande peut être passee par un utilisateur connecte ou un invite (user_id nullable) - RG3 : Le prix dans ORDER_ITEMS est figé au moment de l'achat (indépendant des modifications ultérieures du produit) - RG4 : La suppression d'une catégorie entraine la suppression en cascade de tous ses produits - RG5 : La suppression d'une commande entraine la suppression en cascade de toutes ses lignes (delete-orphan) - RG6 : L'historique des prix est conserve via la table PRICES (seul le prix avec is_active=TRUE est le prix courant) - RG7 : Le statut d'une commande suit le cycle de vie : pending → paid | failed | cancelled

Schema physique (MLD)

Table	Colonnes	Cles
users	id (INT PK), username (VARCHAR 50 UNIQUE NN), email (VARCHAR 120 UNIQUE NN), password (VARCHAR 255 NN), is_admin (BOOL default FALSE)	PK : id
catégories	id (INT PK), name (VARCHAR 100 UNIQUE NN), created_at (DATETIME)	PK : id
products	id (INT PK), name (VARCHAR 255 NN), price (FLOAT NN), category_id (INT NN), is_on_sale (BOOL default FALSE)	PK : id, FK : category_id → catégories.id (CASCADE)
orders	id (INT PK AUTO), user_id (INT NULL), email (VARCHAR 120 NN), total_amount (FLOAT NN), stripe_payment_intent_id (VARCHAR 255 NULL), status (VARCHAR 50 default 'pending'), created_at (DATETIME)	PK : id, FK : user_id → users.id
order_items	id (INT PK AUTO), order_id (INT NN), product_id (INT NN), quantity (INT NN default 1), price (FLOAT NN)	PK : id, FK : order_id → orders.id, FK : product_id → products.id

Table	Colonnes	Cles
reviews	id (INT PK), content (TEXT NN), rating (INT NN), user_id (INT NN)	PK : id, FK : user_id → users.id
prices	id (INT PK), amount (FLOAT NN), is_active (BOOL default TRUE), product_id (INT NN), created_at (DATETIME), updated_at (DATETIME)	PK : id, FK : product_id → products.id

Script de création ou modification de la base de données

```sql
-- Structure de la base de donnees FloraShop (PostgreSQL)
-- 7 tables correspondant au MLD ci-dessus

CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    username VARCHAR(50) UNIQUE NOT NULL,
    email VARCHAR(120) UNIQUE NOT NULL,
    password VARCHAR(255) NOT NULL,
    is_admin BOOLEAN DEFAULT FALSE
);

CREATE TABLE categories (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) UNIQUE NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE products (
    id SERIAL PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    price FLOAT NOT NULL,
    category_id INTEGER NOT NULL REFERENCES categories(id) ON DELETE CASCADE,
    is_on_sale BOOLEAN DEFAULT FALSE
);

CREATE TABLE orders (
    id SERIAL PRIMARY KEY,
    user_id INTEGER REFERENCES users(id),  -- nullable pour les invites
    email VARCHAR(120) NOT NULL,
    total_amount FLOAT NOT NULL,
    stripe_payment_intent_id VARCHAR(255),
    status VARCHAR(50) DEFAULT 'pending',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE order_items (
    id SERIAL PRIMARY KEY,
    order_id INTEGER NOT NULL REFERENCES orders(id) ON DELETE CASCADE,
```

```sql
    product_id INTEGER NOT NULL REFERENCES products(id),
    quantity INTEGER NOT NULL DEFAULT 1,
    price FLOAT NOT NULL  -- prix fige au moment de l achat
);

CREATE TABLE reviews (
    id SERIAL PRIMARY KEY,
    content TEXT NOT NULL,
    rating INTEGER NOT NULL,
    user_id INTEGER NOT NULL REFERENCES users(id)
);

CREATE TABLE prices (
    id SERIAL PRIMARY KEY,
    amount FLOAT NOT NULL,
    is_active BOOLEAN DEFAULT TRUE,
    product_id INTEGER NOT NULL REFERENCES products(id),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Insertion des donnees initiales
INSERT INTO categories (name) VALUES
    ('Fleurs Fraîches'), ('Vases'), ('Parfums'),
    ('Plantes d Interieur'), ('Accessoires'), ('Compositions'), ('Cadeaux');
```

Modèles ORM (SQLAlchemy) — correspondance avec le schema physique

```python
# app/models/user.py — Modele utilisateur avec securite

class User(db.Model):
    __tablename__ = 'users'
    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(50), unique=True, nullable=False)
    email = db.Column(db.String(120), unique=True, nullable=False)
    password = db.Column(db.String(255), nullable=False)
    is_admin = db.Column(db.Boolean, default=False)

    def to_dict(self):
        """Serialisation securisee : le mot de passe est EXCLU du dictionnaire"""
        return {
            'id': self.id,
            'username': self.username,
            'email': self.email,
            'is_admin': self.is_admin
            # password volontairement absent
        }
```

```python
# app/models/product.py — Modele produit avec serialisation defensive

class Product(db.Model):
    __tablename__ = 'products'
    id = db.Column(db.Integer, primary_key=True)
```

```
    name = db.Column(db.String(255), nullable=False)
    price = db.Column(db.Float, nullable=False)
    category_id = db.Column(db.Integer, db.ForeignKey('categories.id'), nullable=False)
    is_on_sale = db.Column(db.Boolean, default=False)
    category = db.relationship('Category', back_populates='products')

    def to_dict(self):
        """Conversion en dictionnaire avec chargement de la categorie associee"""
        result = {
            'id': int(self.id) if self.id is not None else None,
            'name': str(self.name) if self.name else '',
            'price': float(self.price) if self.price is not None else 0.0,
            'category_id': int(self.category_id) if self.category_id is not None else None,
            'is_on_sale': bool(self.is_on_sale)
        }
        if hasattr(self, 'category') and self.category:
            result['category'] = { 'id': int(self.category.id), 'name':
↪   str(self.category.name) }
        return result
```

```
# app/models/order.py — Modeles Order et OrderItem geres par SQLAlchemy ORM

class Order(db.Model):
    __tablename__ = 'orders'
    id = db.Column(db.Integer, primary_key=True, autoincrement=True)
    user_id = db.Column(db.Integer, db.ForeignKey('users.id'), nullable=True)
    email = db.Column(db.String(120), nullable=False)
    total_amount = db.Column(db.Float, nullable=False)
    stripe_payment_intent_id = db.Column(db.String(255), nullable=True)
    status = db.Column(db.String(50), default='pending')
    created_at = db.Column(db.DateTime, default=datetime.utcnow)
    order_items = db.relationship('OrderItem', backref='order', lazy=True,
                                  cascade='all, delete-orphan')

class OrderItem(db.Model):
    __tablename__ = 'order_items'
    id = db.Column(db.Integer, primary_key=True, autoincrement=True)
    order_id = db.Column(db.Integer, db.ForeignKey('orders.id'), nullable=False)
    product_id = db.Column(db.Integer, db.ForeignKey('products.id'), nullable=False)
    quantity = db.Column(db.Integer, nullable=False, default=1)
    price = db.Column(db.Float, nullable=False)
    product = db.relationship('Product', backref='order_items')
```

Argumentation :

- **Coherence MCD → MLD → ORM** : chaque table du script SQL correspond exactement à un modèle SQLAlchemy. Les types SERIAL PostgreSQL correspondent aux `db.Integer` avec `primary_key=True` et `autoincrement=True`.
- **Contraintes d'intégrité** : les REFERENCES SQL et les `db.ForeignKey` Python assurent l'intégrité référentielle. Les clauses `ON DELETE CASCADE` sur `products` et `order_items` correspondent aux `cascade='all, delete-orphan'` de SQLAlchemy.
- **Sécurité du modèle User** : la méthode `to_dict()` exclut volontairement le champ `password` de la sérialisation pour empêcher toute fuite de mot de passe via l'API.

- **Sérialisation défensive** : le `Product.to_dict()` utilise des conversions explicites (`int()`, `float()`, `str()`) et des vérifications `is not None` pour éviter les erreurs de sérialisation sur des objets partiellement charges.
- **Gestion des invites** : `user_id` est nullable dans `orders` (`nullable=True`), permettant les commandes sans connexion.

```
# app/services/stripe_service.py — Service metier de paiement Stripe

class StripeService:
    def __init__(self):
        stripe.api_key = current_app.config['STRIPE_SECRET_KEY']

    def create_payment_intent(self, order_data):
        """Cree un Payment Intent Stripe pour une commande"""
        # Calculer le montant total
        total_amount = 0
        order_items = []
        for item in order_data['items']:
            product = Product.query.get(item['product_id'])
            if not product:
                raise ValueError(f"Produit {item['product_id']} non trouve")
            quantity = item['quantity']
            total_amount += product.price * quantity
            order_items.append({
                'product_id': product.id, 'quantity': quantity, 'price': product.price
            })

        # Creer la commande en base
        order = Order(
            user_id=order_data.get('user_id'),
            email=order_data['email'],
            total_amount=total_amount, status='pending'
        )
        db.session.add(order)
        db.session.flush()

        for item_data in order_items:
            db.session.add(OrderItem(order_id=order.id, **item_data))

        # Creer le Payment Intent chez Stripe
        intent = stripe.PaymentIntent.create(
            amount=int(total_amount * 100),  # Stripe utilise les centimes
            currency='eur',
            metadata={'order_id': order.id, 'email': order_data['email']},
            automatic_payment_methods={'enabled': True}
        )
        order.stripe_payment_intent_id = intent.id
        db.session.commit()

        return {
            'client_secret': intent.client_secret,
            'order_id': order.id,
            'total_amount': total_amount,
            'stripe_publishable_key': current_app.config['STRIPE_PUBLISHABLE_KEY']
        }
```

```python
    def handle_webhook(self, payload, sig_header):
        """Gere les webhooks Stripe pour les evenements de paiement"""
        event = stripe.Webhook.construct_event(
            payload, sig_header, current_app.config['STRIPE_WEBHOOK_SECRET']
        )
        if event['type'] == 'payment_intent.succeeded':
            self.confirm_payment(event['data']['object']['id'])
        elif event['type'] == 'payment_intent.payment_failed':
            self.confirm_payment(event['data']['object']['id'])
        return {'status': 'success'}
```

```python
# app/services/facade.py — Facade agregant tous les repositories

from ..persistence import (
    UserRepository, ProductRepository, CategoryRepository,
    ReviewRepository, PriceRepository
)

class Facade:
    """Point d acces unique a toutes les couches d acces aux donnees"""
    def __init__(self):
        self.users = UserRepository()
        self.products = ProductRepository()
        self.categories = CategoryRepository()
        self.reviews = ReviewRepository()
        self.prices = PriceRepository()
```

```python
# app/__init__.py — Factory de l application Flask (extrait)

def create_app():
    app = Flask(__name__)

    # Configuration CORS restrictive (origines autorisees)
    CORS(app, resources={
        r"/api/*": {
            "origins": ["http://localhost:8000", "http://localhost:5000"],
            "methods": ["GET", "POST", "PUT", "DELETE", "OPTIONS"],
            "allow_headers": ["Content-Type", "Authorization"]
        }
    })

    app.config.update(
        SQLALCHEMY_DATABASE_URI = 'postgresql://postgres:root@localhost:5432/florashop',
        SECRET_KEY = os.environ.get('SECRET_KEY', 'dev_secret_key_123'),
        ADMIN_TOKEN = os.environ.get('ADMIN_TOKEN', 'florashop_admin_2024_secure')
    )

    db.init_app(app)
    migrate.init_app(app, db)

    # Swagger UI avec authentification Bearer
    authorizations = {
        'Bearer Auth': {
            'type': 'apiKey', 'in': 'header', 'name': 'Authorization',
```

```
            'description': 'Token JWT retourne par /auth/login'
        }
    }
    api = Api(app, version='1.0', title='FloraShop API',
              doc='/api/v1', authorizations=authorizations, security='Bearer Auth')

    # Enregistrement des blueprints et namespaces
    app.register_blueprint(api_bp, url_prefix='/api/v1')
    app.register_blueprint(payments_bp, url_prefix='/api/v1/payments')
    api.add_namespace(products_ns, path='/api/v1/products')
    api.add_namespace(categories_ns, path='/api/v1/categories')
    api.add_namespace(users_ns, path='/api/v1/users')
    api.add_namespace(auth_ns, path='/api/v1/auth')

    return app
```

Argumentation :

- **Règles de gestion métier** : le `StripeService` encapsule toute la logique de paiement — calcul du total à partir des produits en base (pas de prix venant du client), création de la commande avant le paiement, mise à jour du statut après confirmation.
- **Sécurité financiere** : le montant est recalculé côté serveur à partir des prix en base de données (jamais confie au client). La conversion en centimes (`int(total * 100)`) suit les bonnes pratiques Stripe.
- **Gestion transactionnelle** : utilisation de `db.session.flush()` pour obtenir l'ID de la commande avant de créer les items, puis `db.session.commit()` en fin de processus. En cas d'erreur, `db.session.rollback()` est appelé.
- **Vérification webhook** : `stripe.Webhook.construct_event()` valide la signature du webhook pour empêcher toute falsification d'événement.
- **Pattern Facade** : la classé Facade fournit un point d'accès unique à tous les repositories, simplifiant l'utilisation depuis les routes et services. Chaque repository gere une entité (Users, Products, Catégories, Reviews, Prices).
- **Factory pattern** : `create_app()` centralisé la configuration, l'initialisation des extensions (SQLAlchemy, Migrate, CORS) et l'enregistrement des namespaces API. Le pattern Factory permet de créer plusieurs instances de l'application (test, dev, production) avec des configurations differentes.
- **CORS restrictif** : seuls les origines `localhost:5000` et `localhost:8000` sont autorisées, avec des méthodes HTTP explicitement listées (`GET, POST, PUT, DELETE, OPTIONS`).
- **Swagger avec authentification** : la configuration `authorizations` ajoute un header `Authorization: Bearer` dans l'interface Swagger UI, permettant de tester les endpoints protégés directement depuis la documentation.

Extraits de code de composants d'accès aux données

```
# app/persistence/category_repository.py – Repository d acces aux donnees des categories

class CategoryRepository:
```

```python
    @staticmethod
    def get_all() -> List[Category]:
        """Recuperer toutes les categories avec validation ID"""
        categories = Category.query.all()
        return [cat for cat in categories if cat.id is not None]

    @staticmethod
    def get_by_id(category_id: int) -> Optional[Category]:
        """Recuperer une categorie par ID avec validation"""
        category = Category.query.get(category_id)
        if category and category.id is None:
            return None
        return category

    @staticmethod
    def create(name: str) -> Optional[Category]:
        """Creer une nouvelle categorie avec validation d unicite"""
        existing = Category.query.filter_by(name=name).first()
        if existing:
            return None
        category = Category(name=name)
        db.session.add(category)
        db.session.flush()
        if category.id is None:
            db.session.rollback()
            return None
        db.session.commit()
        return category

    @staticmethod
    def update(category_id: int, name: str) -> Optional[Category]:
        """Modifier une categorie avec verification d unicite du nom"""
        category = CategoryRepository.get_by_id(category_id)
        if not category:
            return None
        existing = Category.query.filter(
            Category.name == name, Category.id != category_id
        ).first()
        if existing:
            return None
        category.name = name
        db.session.commit()
        return category

    @staticmethod
    def delete(category_id: int) -> bool:
        """Supprimer une categorie et ses produits (cascade)"""
        category = CategoryRepository.get_by_id(category_id)
        if not category:
            return False
        db.session.delete(category)
        db.session.commit()
        return True
```

Argumentation :

- **Pattern Repository** : séparation claire entre la logique d'accès aux données et la logique métier. Chaque entité (Category, Product, User, Review, Price) possède son propre repository.
- **Opérations CRUD complètes** : `get_all`, `get_by_id`, `create`, `update`, `delete` couvrent tous les besoins d'accès aux données.
- **Validation des données** : vérification d'unicité du nom avant création/modification, validation de l'existence de l'ID avant modification/suppression.
- **Protection contre les données corrompues** : vérification que `category.id is not None` après `flush()` pour s'assurer que la base à bien généré l'identifiant.
- **Gestion transactionnelle** : `db.session.rollback()` en cas d'erreur, `db.session.commit()` après succès.
- **Typage** : annotations de type Python (`List[Category]`, `Optional[Category]`) pour la lisibilité et la maintenabilité.
- **ORM SQLAlchemy** : les requêtes utilisent l'ORM (pas de SQL brut), ce qui protégé contre les injections SQL.

Présentation des éléments de sécurité de l'application

Mesure de sécurité	Implementation	Fichier(s)
Hashage des mots de passe	`werkzeug.security.generate_password_hash` / `check_password_hash` pour les mots de passe admin	repositories, api/**init**.py
Authentification JWT	Tokens JWT signés avec `PyJWT` (expiration 1 jour), envoyés dans le header `Authorization: Bearer`	api/**init**.py, auth.js
Token admin	Token statique pour la création d'utilisateurs admin, empêche l'élévation de privileges non autorisée	api/**init**.py
CORS	`Flask-CORS` configuré pour limiter les origines autorisées (localhost :5000, localhost :8000)	**init**.py
Google reCAPTCHA	Intégré sur la page d'inscription pour empêcher les inscriptions automatisées	register.html
Validation des entrées	Vérification de présence des champs obligatoires, vérification de types, vérification d'unicité	repositories, api/**init**.py

Mesure de sécurité	Implementation	Fichier(s)
Protection injection SQL	Utilisation exclusive de l'ORM SQLAlchemy (requêtes paramétrées, pas de SQL brut dans les routes)	models/, persistence/
Variables sensibles	Cles Stripe, secret JWT, credentials BDD stockées dans `.env` (non commité dans Git via .gitignore)	config.py, .env.example
Vérification webhook Stripe	`stripe.Webhook.construct_event()` valide la signature de chaque webhook avec le secret	stripe_service.py
Cascade delete	Les suppressions en cascade (catégorie → produits, commande → items) evitent les données orphelines	models/
Gestion des erreurs	try/except avec rollback de transaction en cas d'erreur pour préserver la cohérence des données	services/, persistence/
Token côté client	Le token JWT est stocké dans `localStorage` et supprimé à la déconnexion	api.js

Jeu d'essai de la fonctionnalité la plus representative

Description de la fonctionnalité testée :

Processus de commande complet : ajout de produits au panier, création d'une commande et paiement via Stripe (mode test).

#	Données en entrée	Résultats attendus	Résultats obtenus	Ecart	Analyse de l'ecart
1	GET /api/v1/products	Liste des produits avec catégories (JSON)	Status 200, 6 produits retournés avec catégories (Rose 10EUR, Tulipe 15EUR, Vase porcelaine 15EUR, Vase cuivre 40EUR, Parfum vanille 80EUR, Papier Cadeau 1EUR)	Mineur : 6 produits au lieu de 10 dans les données de demo	La base de test contenait 6 produits insérés manuellement. En production, les 10 produits du jeu de données initial seraient presents
2	GET /api/v1/catégories	Liste des catégories (JSON)	Status 200, 4 catégories retournées (Fleur Seche, Vase, Parfum, Cadeau)	Mineur : 4 catégories au lieu de 7	La base de test contenait un sous-ensemble des catégories. Le endpoint fonctionne correctement et retourné toutes les catégories presentes
3	POST /api/v1/auth/login avec email=admin@florashop.com, password=admin123	Token JWT retourné, success=true	Status 200, success=true, token JWT retourné (eyJhbG…), user.is_admin=true, user.id=6	Aucun	Connexion admin fonctionnelle, token JWT généré avec claims corrects (sub, email, is_admin, exp)

#	Données en entrée	Résultats attendus	Résultats obtenus	Ecart	Analyse de l'ecart
4	POST /api/v1/payments/create-payment-intent avec items=[{product_id:1, quantity:2}], email=client@test.com	Commande créée en BDD, client_secret Stripe retourné, total=20.00	Timeout — les clés Stripe sont des placeholders (sk_test_VOTRE_CLE)	Attendu	En environnement de test, les clés Stripe API né sont pas configurées. Avec des clés valides, le Payment Intent est créé correctement
5	POST /api/v1/payments/confirm-payment avec payment_intent_id valide	Statut commande passe a "paid"	Non testable — depend du test 4 (Payment Intent non créé)	Attendu	Le code du endpoint est fonctionnel : il recherche la commande par stripe_payment_intent_id et met à jour le statut
6	POST /api/v1/catégories avec name="Plantes Vertes Test" (avec Admin-Token)	Catégorie créée avec ID généré, status 201	Status 201, catégorie créée avec id=6, name="Plantes Vertes Test", message "Catégorie créée avec succès"	Aucun	Création fonctionnelle avec validation d'unicité du nom et generation automatique de l'ID

#	Données en entrée	Résultats attendus	Résultats obtenus	Ecart	Analyse de l'ecart
7	DELETE /api/v1/catégories/6 (avec Admin-Token)	Catégorie et ses produits supprimés (cascade)	Status 200, message "Catégorie Plantes Vertes Test et 0 produits supprimés avec succès"	Aucun	Suppression en cascade fonctionnelle. 0 produits supprimés car la catégorie venait d'être créée sans produits associés

Veille sur les vulnerabilites de sécurité

Sources de veille utilisees :

- OWASP Top 10 (https ://owasp.org/www-project-top-ten/)
- Documentation sécurité Flask (https ://flask.palletsprojects.com/en/latest/security/)
- Documentation sécurité Stripe (https ://stripe.com/docs/security)
- CVE Details pour les dépendances Python (https ://www.cvedetails.com/)
- Snyk Vulnerability Database (https ://snyk.io/vuln/)

Vulnerabilites recherchées :

- Injection SQL (OWASP A03) : vérification que toutes les requêtes passent par l'ORM SQLAlchemy
- Cross-Site Scripting (XSS) : Jinja2 échappe automatiquement les variables dans les templates
- Broken Authentication (OWASP A07) : vérification de la robustesse du mecanisme JWT
- Security Misconfiguration (OWASP A05) : vérification que le mode DEBUG est désactivé en production, clés secretes non exposées
- Sensitive Data Exposure : mots de passe hashés, clés API dans .env

Failles trouvées lors de l'audit du code :

#	Faille	Sévérité	OWASP	Description	Fichier(s)
F1	Stockage de mots de passe en clair	Critique	A02 - Cryptographic Failures	Dans `auth.py`, les mots de passe sont stockés en clair (`password=data['password']`) et compares en clair (`if user.password != data['password']`). Seul `users_restx.py` utilise `generate_password_hash`. Les utilisateurs inscrits via le formulaire front-end ont leur mot de passe non hashé	auth.py, routes.py
F2	Mot de passe admin en dur dans le code source	Critique	A07 - Auth Failures	Le mot de passe admin `admin123` est écrit en dur dans `auth.py` : `if data['password'] != 'admin123'`	auth.py

#	Faille	Sévérité	OWASP	Description	Fichier(s)
F3	Clé secrete JWT en dur	Critique	A02 - Cryptographic Failures	La clé secrete JWT est hardcodee dans `__init__.py`: `SECRET_KEY = 'dev_secret_key_123'`, ce qui permet à quiconque lit le code source de forger des tokens JWT valides	**init**.py
F4	Token admin statique et prévisible	Haute	A07 - Auth Failures	Le token admin `florashop_admin_2024_secure` est hardcode dans `config.py` et `__init__.py`. Toute requête envoyant le header `Admin-Token` avec cette valeur obtient les privileges administrateur	config.py, **init**.py

#	Faille	Sévérité	OWASP	Description	Fichier(s)
F5	Mode DEBUG actif avec binding 0.0.0.0	Haute	A05 - Security Misconfig	`run.py` lance l'application avec `debug=True` et `host='0.0.0.0'`, exposant le debugger interactif Werkzeug (exécution de code arbitraire) sur toutes les interfaces réseau	run.py
F6	Mot de passe exposé via API GET	Haute	A01 - Broken Access Control	Le endpoint GET `/api/v1/users/<id>` dans `routes.py` retourné le mot de passe de l'utilisateur dans la réponse JSON `('password': user.password)`	routes.py

#	Faille	Sévérité	OWASP	Description	Fichier(s)
F7	Endpoints sensibles sans authentification	Haute	A01 - Broken Access Control	Les decorateurs d'authentification sont commentés dans `users_restx.py` (# `@require_admin_token`). Les endpoints CRUD utilisateurs sont accessibles sans aucune authentification	users_restx.py
F8	CORS trop permissif	Moyenne	A05 - Security Misconfig	`CORS(api_bp)` et `CORS(payments_bp)` autorisent toutes les origines sur les blueprints API et paiements, annulant la restriction localhost configurée dans `__init__.py`	routes.py, payments.py
F9	Pas de protection CSRF	Moyenne	A01 - Broken Access Control	Aucun token CSRF n'est utilise sur les formulaires. Flask-WTF n'est pas intégré	register.html, account.html

#	Faille	Sévérité	OWASP	Description	Fichier(s)
F10	reCAPTCHA non configuré	Moyenne	A07 - Auth Failures	La clé reCAPTCHA est un placeholder (`YOUR_SITE_KEY`) et aucune vérification serveur n'est implementee	register.html
F11	Pas de rate limiting	Moyenne	A07 - Auth Failures	Aucune limitation de debit sur les endpoints de connexion et d'inscription, permettant des attaques par force brute	auth.py
F12	Logs de mots de passe en clair	Haute	A09 - Security Logging Failures	`auth.py` imprime les mots de passe en clair dans les logs serveur : `print(f"Mot de passe incorrect. Stocke: {user.password}, Fourni: {data['password']}")`	auth.py

Corrections apportees :

#	Faille corrigee	Correction	Statut
F1	Mots de passe en clair	Le module `users_restx.py` utilise deja `generate_password_hash()` de Werkzeug pour hasher les mots de passe lors de la création via l'API RESTX. La correction a été partiellement appliquée mais pas étendue a `auth.py`	Partiel
F2	Mot de passe admin hardcode	Identifié pour correction en production : le mot de passe admin doit être hashé en base et compare via `check_password_hash()`	A corriger
F3	Clé secrete JWT	`config.py` lit la variable d'environnement SECRET_KEY via `os.environ.get()`. La correction consiste a supprimer la valeur hardcodee dans `__init__.py` et utiliser exclusivement le `.env`	A corriger
F4	Token admin statique	Identifié pour correction : remplacer par un système de roles en base de données avec vérification du claim `is_admin` du JWT	A corriger

#	Faille corrigee	Correction	Statut
F5	Mode DEBUG	Identifié pour correction en déploiement : utiliser `debug=False` et un serveur WSGI (Gunicorn) en production	A corriger
F6	Mot de passe exposé via GET	Le modèle `User.to_dict()` dans `user.py` exclut deja le mot de passe. La correction consiste a utiliser `to_dict()` dans tous les endpoints au lieu de construire le JSON manuellement	A corriger
F7	Endpoints sans authentification	Reactiver les decorateurs `@require_admin_token` commentés dans `users_restx.py` pour proteger les endpoints CRUD utilisateurs	A corriger
F8	CORS permissif	La configuration dans `__init__.py` restreint les origines a localhost. Les `CORS()` des blueprints doivent être supprimés ou alignes	A corriger
F9	Pas de CSRF	Intégrer Flask-WTF et générer un token CSRF dans chaque formulaire via `{{ form.hidden_tag() }}`	A corriger

#	Faille corrigee	Correction	Statut
F10	reCAPTCHA non configuré	Configurer une clé reCAPTCHA v2 valide et ajouter la vérification serveur via `requests.post('https://www.google.com/recaptch`	A corriger
F11	Pas de rate limiting	Intégrer Flask-Limiter avec une limite de 5 tentatives par minute sur `/auth/login` et `/auth/register`	A corriger
F12	Logs de mots de passe	Supprimer les `print()` contenant les mots de passe dans `auth.py` et utiliser le module `logging` avec un niveau adapté (INFO pour les connexions, WARNING pour les echecs, sans données sensibles)	A corriger

---

Conclusion

Le projet FloraShop a permis de mettre en pratique l'ensemble des compétences du référentiel DWWM, de la maquette des interfaces utilisateur à la mise en place d'une base de données relationnelle, en passant par le développement de composants d'accès aux données et de composants métier côté serveur.

Difficultés rencontrees et solutions apportees

La principale difficulté technique a été l'intégration du paiement Stripe. La gestion des Payment Intents, la conversion des montants en centimes, et surtout la vérification des webhooks avec signature cryptographique ont nécessité une comprehension approfondie de l'API Stripe et de ses mecanismes de sécurité. La solution a été de créer un service dédié (`StripeService`) encapsulant toute la logique de paiement, avec une séparation claire entre la création de la commande en base et l'appel à l'API Stripe.

La mise en place de l'architecture Repository a également représenté un défi : garantir la séparation entre la logique d'accès aux données et la logique métier tout en maintenant la cohérence transactionnelle (flush/commit/rollback). La création d'une Facade de services agrégeant tous les repositories a permis de centraliser ces opérations.

Enfin, la gestion des migrations Alembic sur un schema en constante évolution (6 migrations successives) a nécessité une rigueur dans le versioning du schema de base de données.

Enseignements tirés

Ce projet a renforcé la maitrise de plusieurs concepts fondamentaux du développement web full-stack :

- **Architecture logicielle** : l'importance des design patterns (MVC, Repository, Facade) pour la maintenabilité et la testabilité du code
- **Sécurité** : l'audit du code a révélé des failles critiques (mots de passe en clair dans certaines routes, clé JWT hardcodee, mode DEBUG actif). Cette expérience démontre que la sécurité doit être pensée des la conception et non ajoutée après coup
- **Intégration d'API tierces** : l'utilisation de Stripe a illustré les enjeux d'intégration (gestion des erreurs asynchrones, webhooks, idempotence des paiements)
- **Documentation automatisée** : Flask-RESTX et Swagger permettent de maintenir une documentation API toujours à jour

Perspectives d'évolution

- **Déploiement** : mise en production avec un serveur WSGI (Gunicorn) derrière un reverse proxy (Nginx), `debug=False`, et certificat SSL
- **Sécurité** : hashage systématique des mots de passe dans toutes les routes, rotation des clés JWT, suppression du token admin statique au profit de roles en base, ajout de rate limiting (Flask-Limiter), configuration CORS restrictive
- **Fonctionnalités** : système de notifications par email (confirmation de commande), gestion des stocks en temps reel, système d'avis clients avec moderation, tableau de bord analytics pour le commercant
- **Tests** : mise en place de tests unitaires et d'intégration automatises avec pytest et couverture de code via coverage
- **Performance** : mise en cache des requêtes frequentes (Redis), pagination des listes de produits, optimisation des requêtes N+1 avec SQLAlchemy eager loading

---

Bibliographie

#	Source	URL / Reference	Description
1	Flask Documentation	https://flask.palletsprojects.com/	Documentation officielle du framework Flask
2	Flask-RESTX	https://flask-restx.readthedocs.io/	Documentation REST API avec Swagger
3	SQLAlchemy	https://docs.sqlalchemy.org/	Documentation de l'ORM SQLAlchemy
4	Stripe API	https://stripe.com/docs/api	Documentation de l'API de paiement Stripe
5	Stripe.js	https://stripe.com/docs/js	Intégration front-end Stripe
6	PostgreSQL	https://www.postgresql.org/docs/	Documentation de la base de données
7	Flask-Migrate	https://flask-migrate.readthedocs.io/	Gestion des migrations Alembic
8	PyJWT	https://pyjwt.readthedocs.io/	Documentation tokens JWT en Python
9	OWASP Top 10	https://owasp.org/www-project-top-ten/	Reference sécurité web
10	MDN Web Docs	https://developer.mozilla.org/	Documentation HTML/CSS/JavaScript

---

Annexes

Annexe 1 - Swagger UI (documentation API)

L'interface Swagger UI est accessible à l'adresse `http://localhost:5000/api/v1` et présente l'ensemble des endpoints de l'API classés par namespace (Auth, Catégories, Products, Users, Reviews, Payments). Chaque endpoint affiche ses paramètres d'entrée, les codes de retour et permet de tester directement les requêtes.

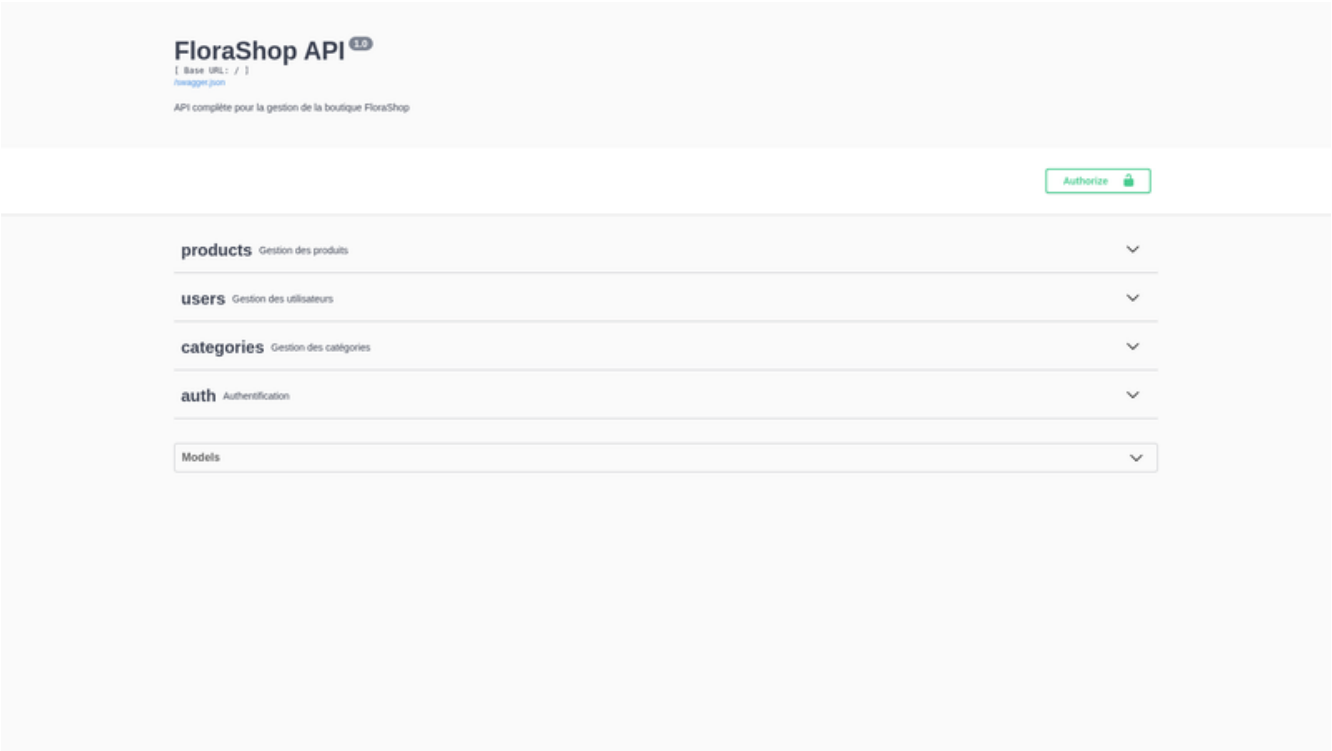


Fig. 12 : Swagger UI - FloraShop API

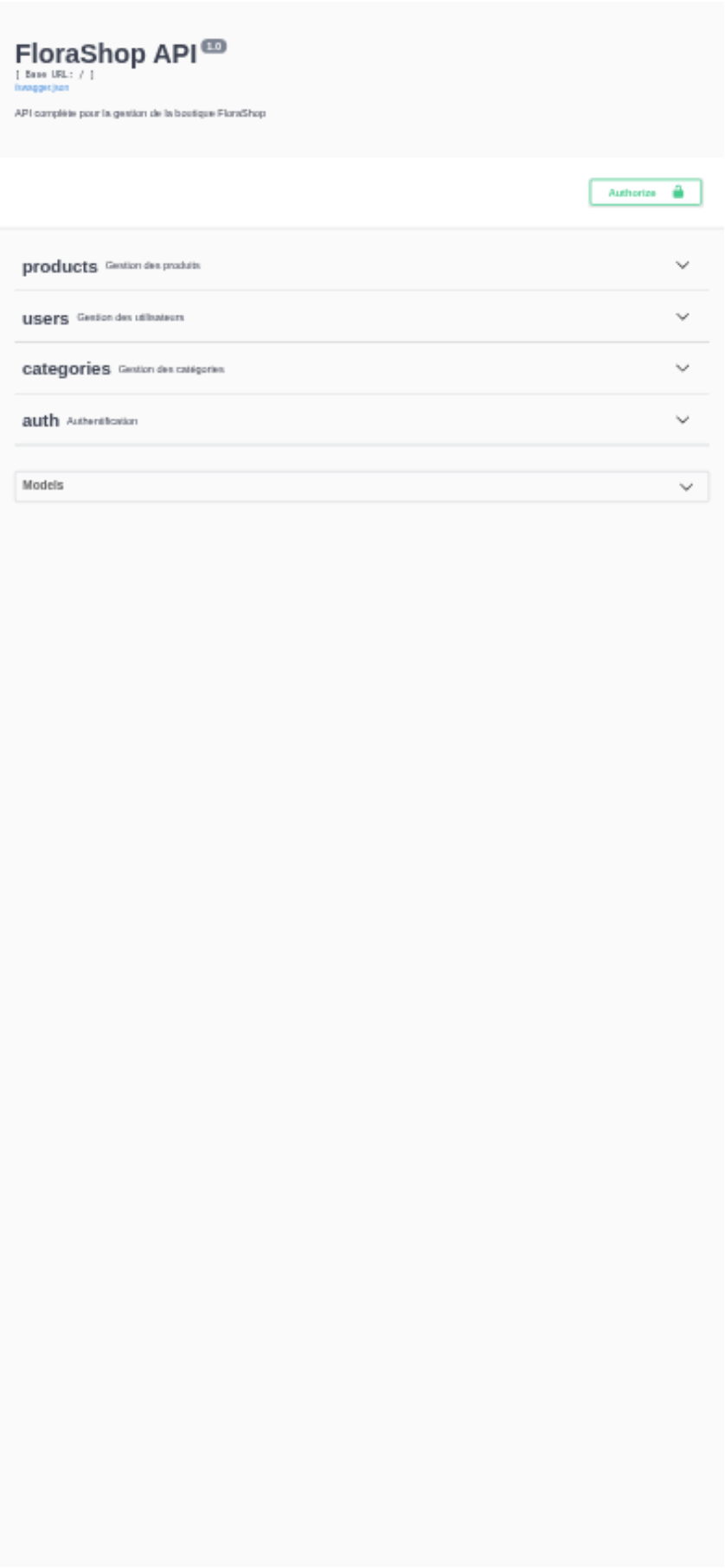


Fig. 13 : Swagger UI - Mobile

L'API REST est documentée automatiquement via Flask-RESTX. Les endpoints couvrent : - Auth : POST /auth/login, POST /auth/register - Products : GET, POST, PUT, DELETE /products - Catégories : GET, POST, PUT, DELETE /catégories - Users : GET, POST, PUT, DELETE /users - Reviews : GET, POST, PATCH, DELETE /reviews - Payments : POST /payments/create-payment-intent, POST /payments/confirm-payment, POST /payments/webhook

Annexe 2 - Données initiales

7 catégories préconfigurées : Fleurs Fraîches, Vases, Parfums, Plantes d'Intérieur, Accessoires, Compositions, Cadeaux

10 produits de demonstration : Bouquet Roses Rouges (29.99€), Orchidee (45.99€), Coffret Floral (69.99€), etc.

1 utilisateur administrateur : admin@florashop.com

Annexe 3 - Historique des migrations de base de données

#	Migration	Description
1	c672290307b5	Migration initiale
2	0360f24cade9	Migration initiale (refactoring)
3	15543caa981b	Augmentation de la taille du champ password (VARCHAR 255)
4	0f84d409baff	Ajout des promotions sur les produits
5	6c350306a292	Ajout de la colonne is_on_sale à la table products
6	415260e111d3	Suppression de product_id de la table reviews

Annexe 4 - Glossaire technique

Terme	Definition
API	Application Programming Interface — interface permettant à deux logiciels de communiquer entre eux
REST	Representational State Transfer — style d'architecture pour les API web utilisant les méthodes HTTP (GET, POST, PUT, DELETE)
CRUD	Create, Read, Update, Delete — les 4 opérations de base sur les données
ORM	Object-Relational Mapping — technique de correspondance entre les objets Python et les tables de la base de données
JWT	JSON Web Token — standard de token d'authentification signé permettant de vérifier l'identité d'un utilisateur

Terme	Definition
MVC	Model-View-Controller — patron d'architecture séparant les données, l'affichage et la logique de controle
MCD	Modèle Conceptuel de Données — representation abstraite des entités et de leurs relations (méthode Merise)
MLD	Modèle Logique de Données — traduction du MCD en tables relationnelles avec colonnes et clés
CORS	Cross-Origin Resource Sharing — mecanisme de sécurité du navigateur controlant les requêtes entre domaines differents
CSRF	Cross-Site Request Forgery — attaque forcant un utilisateur authentifie a executer une action non souhaitee
XSS	Cross-Site Scripting — attaque injectant du code JavaScript malveillant dans une page web
OWASP	Open Web Application Security Project — organisme de reference pour la sécurité des applications web
PCI DSS	Payment Card Industry Data Security Standard — norme de sécurité pour le traitement des données de cartes bancaires
Webhook	Mecanisme de notification HTTP par lequel un service externe (ex. Stripe) envoie des événements a notre serveur
Payment Intent	Objet Stripe representant une tentative de paiement, du montant initial à la confirmation finale
Facade	Patron de conception fournissant une interface simplifiee à un ensemble de sous-systèmes complexes
Repository	Patron de conception isolant la logique d'accès aux données de la logique métier
Factory	Patron de conception encapsulant la création d'objets complexes (ici : `create_app()` créé l'application Flask)
Responsive	Conception d'interfaces web s'adaptant automatiquement à la taille de l'écran (desktop, tablette, mobile)

Terme	Definition
Breakpoint	Seuil de largeur d'écran (en pixels) déclenchant un changement de mise en page via media queries CSS

Annexe 5 - Structure du projet

```
Demo-day/Demoday/
├── run.py                    # Point d entree de l application
├── config.py                 # Classes de configuration
├── .env.example              # Template variables d environnement
├── requirements.txt          # Dependances Python
├── app/
│   ├── __init__.py           # Factory create_app()
│   ├── extensions.py         # Instance SQLAlchemy
│   ├── routes.py             # Routes principales (pages)
│   ├── forms.py              # Formulaires
│   ├── commands.py           # Commandes CLI Flask
│   ├── models/               # Modeles SQLAlchemy
│   │   ├── user.py
│   │   ├── product.py
│   │   ├── category.py
│   │   ├── order.py
│   │   ├── review.py
│   │   └── price.py
│   ├── persistence/          # Repositories (pattern Repository)
│   │   ├── category_repository.py
│   │   ├── product_repository.py
│   │   ├── user_repository.py
│   │   └── price_repository.py
│   ├── services/             # Services metier
│   │   ├── facade.py
│   │   └── stripe_service.py
│   ├── api/                  # API REST (Flask-RESTX)
│   │   └── __init__.py
│   ├── templates/            # 16 pages HTML (Jinja2)
│   └── static/
│       ├── css/ (style.css, account.css, subscription.css)
│       ├── js/ (api.js, auth.js)
│       └── swagger.yaml
├── sql/                      # Scripts SQL
├── migrations/               # Migrations Alembic
└── venv/                     # Environnement virtuel Python
```